# Lecture 6 · Optimization Algorithms: From SGD to Muon

**孙若愚　Ruoyu Sun**

The Chinese University of Hong Kong, Shenzhen

ME2605 / DDA6204 · Fall 2026

香港中文大學（深圳）
The Chinese University of Hong Kong , Shenzhen

深圳市大数据研究院
Shenzhen Research Institute of Big Data

深圳国际工业与应用数学中心
SHENZHEN INTERNATIONAL CENTER FOR INDUSTRIAL AND APPLIED MATHEMATICS

# Overview: Eight Sections

- Part 1 SGD and Momentum
- Part 2 From AdaGrad to Adam
- Part 3 Does Adam Converge?
- Part 4 Why Transformers Need Adam
- Part 5 Adam-mini
- Part 6 From Shampoo to Muon
- Part 7 Warmup and Schedules
- Part 8 Summary

Notation: this deck writes $\theta_t$ (or $w_t$) for the iterate at step t. Lectures 3–5 wrote the same object as w[r].

# Background

- **Adam** is the most popular optimizer in neural networks. (**210k** citations, by March 2024)

- Adam 很可能是历史上单篇**最高引**优化算法论文

```
optimizer = optim.Adam(net.parameters(), lr=args.lr, betas=(args.beta1, args.beta2), eps=1e-08,
                       weight_decay=args.weightdecay, amsgrad=False)
```

- Adam 是业界 LLM 训练的**主要选择之一**
  (Muon **emerges recently; will discuss later** )

# Part 1 SGD and Momentum

The two algorithms everything else in this lecture is about.

W at a constant step size is wort , and w ere momentum comes from.

# SGD for Finite Sum

- Consider an optimization problem

$$\min_{w \in \mathbb{R}^d} F(w) \triangleq \sum_{i=1}^{n} F_i(w),$$

  where $F_i$ is differentiable.

- **SGD** (Stochastic Gradient Method): at each iteration $t$, pick a sample $i_t \in [n]$ uniformly at random, then update

$$w_{t+1} = w_t - \alpha_t \nabla F_{i_t}(w^t).$$

# Constant Stepsize SGD on strongly convex functions

Consider SGD defined as follows:

$$w_{k+1} = w_k - \alpha \nabla F(w_k; \xi_k), \tag{1}$$

where $\xi_k$ is a random variable independent of previous $\xi_0, \ldots, \xi_{k-1}$.

**Assumption 1** (unbiased) $E[\nabla F(w_k, \xi_k)] = \nabla F(w_k), \forall k$.
**Assumption 2** (bounded variance) $E\|\nabla F(w_k, \xi_k) - \nabla F(w_k)\|^2 \leq \sigma^2$.

**Thm** 1 For any $L$-smooth and $\mu$-convex function $F$. Assume Assumption 1 and 2 hold, and the stepsize $\alpha \leq 2/(\mu + L)$. Let $\Phi_t = \mathbb{E}\|w_t - w^*\|^2$ and $\Phi^* = \frac{\alpha\sigma^2}{\mu(2+\mu\alpha)}$. Then

$$\Phi_t \leq \Phi^* + (1 - \mu\alpha)^{2t}[\Phi_0 + \alpha^2\sigma^2 - \Phi^*] \tag{2}$$

**Remark**: As $t \to \infty$, $\mathbb{E}\|w_t - w^*\|^2$ converges to $\Phi^* = O(\sigma^2)$.

[Bottou et al. 2018, Thm 4.6] is on the convergence of function value. "Optimization Methods for Large-Scale Machine Learning", SIAM review.

# Averaging Trick on Weights

Averaging (or summing) is common in algorithms.

**Example 1:** Polyak-Ruppert averaging (1988,1990); averaged SGD (ASGD).
Apply averaging to weights (0th order info).

$$\textbf{Regular update}: w_{t+1} = w_t - \alpha \nabla F_{i_t}(w_t), t = 0, 1, \ldots.$$

$$\textbf{Averaging}: \quad \bar{w}_{t+1} = \frac{t}{t+1}\bar{w}_t + \frac{1}{t+1} w_{t+1}, t = 0, 1, \ldots. \tag{1}$$

A variant called EMA is also popular.

$$\textbf{Regular update}: w_{t+1} = w_t - \alpha \nabla F_{i_t}(w_t),$$

$$\textbf{EMA}: \quad \bar{w}_{t+1} = \beta \bar{w}_t + (1-\beta) w_{t+1}. \tag{2}$$

**Claim 1a**: (1) implies $\bar{w}_t = \frac{1}{t}\sum_{i=1}^{t} w_i, t = 1, 2, \ldots.$
**Claim 1b**: (2) and $\bar{w}_{-1} = 0$ implies $\bar{w}_t = (1-\beta)\sum_{i=0}^{t} \beta^{t-i} w_i.$

PyTorch implementation of ASGD: torch.optim.ASGD

# Averaging Trick on Gradients: Momentum

**Example 2:** SGD with Momentum. Let $g_t = F_{i_t}(w_t)$.

$$v_t = \beta v_{t-1} - \alpha \nabla F(\mathbf{w}_t), \quad t = 0, 1, \ldots. \tag{3}$$
$$\mathbf{w}_{t+1} = \mathbf{w}_t + v_t, \quad t = 0, 1, \ldots.$$

**Claim 2**: (3) implies

$$v_t = \beta^{t+1} v_{-1} - \alpha \sum_{k=0}^{t} \beta^{t-i} g_k. \tag{4}$$

When $v_{-1} = 0$, the above is the exponential moving average of the gradients (first-order info) $(\nabla F(w_0), \ldots, \nabla F(w_t))$.

# Part 2 From AdaGrad to Adam

AdaGrad → RMSProp → Adam. Where each step came from, and why AdaGrad, RMSProp and Adam are all Gauss-Newton approximations.

# AdaGrad

Let $\circ$ denote entry-wise product. For a vector $u$, the notation $\sqrt{u}$ and $1/u$ also means entry-wise square root and entry-wise reciprocal.

At the $t$-th iteration, randomly pick $i$, and update the parameter as

$$\begin{aligned} g_t &= \nabla F_i(\theta_t), \\ v_t &= v_{t-1} + g_t \circ g_t, \\ \theta_{t+1} &= \theta_t - \alpha_t \frac{g_t}{\sqrt{v_t} + \epsilon}, \quad t = 0, 1, 2, \ldots, \end{aligned} \tag{5}$$

In other words, the step-size for the $k$-th coordinate is adjusted from $\alpha_t$ in standard SGD to $\alpha_t / \sqrt{\sum_{j=0}^{t} g_{j,k}^2}$ where $g_{j,k}$ denotes the $k$-th entry of $g_j$.

Effectively, $\theta_{t+1} = \theta_t - \alpha_t (\sum_{j=0}^{t} g_j \circ g_j)^{-1/2} \circ g_t$.

Diagnoal preconditioner!

# RMSProp

One drawback of AdaGrad: it treats all samples equally.

Natural idea: use EMA to define $v_t$.

**RMSProp**: At the $t$-th iteration, randomly pick $i$ and compute $g_t = \nabla F_i(\theta_t)$, then:

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t \circ g_t,$$
$$\theta_{t+1} = \theta_t - \alpha_t \frac{g_t}{\sqrt{v_t} + \epsilon}.$$

PyTorch default: $\beta_2 = 0.99, \epsilon = 10^{-8}$ (together with $\alpha_t = 0.01$)
Common choice: $\beta_2 \in \{0.9, 0.99, 0.999\}$.

# Adam

Adam [Kingma and Ba, 2014] is (almost) the combination of RMSProp and momentum.

**Adam**: At the $t$-th iteration of Adam, we randomly pick $i$ and compute $g_t = \nabla F_i(\theta_t)$, then:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad \text{[update 1st order momentum]}$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t \circ g_t, \qquad \text{[update “2nd order” momentum]}$$
$$\theta_{t+1} = \theta_t - \alpha_t \frac{m_t}{\sqrt{v_t} + \epsilon} \qquad \text{[update parameter]} \ .$$

PyTorch default: $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$ (together with $\alpha_t = 0.001$).

learning rate: $\alpha_t$ can be constant, or follow a certain LR schedule.

# Design of Adam: Quasi-Newton Method

- Newton method $\theta_{r+1} = \theta_r - \alpha_r \left(\nabla^2 F(\theta_r)\right)^{-1} \nabla F(\theta_r),\ r = 0, 1, \cdots$

- Instead of exact Hessian, assume we have some way to approximate $\nabla^2 F(\theta_k)^{-1}$, i.e.

$$H_k \approx \nabla^2 F(\theta_k)^{-1}.$$

- A framework of methods using 2nd order information:

$$\theta_{k+1} = \theta_k - \alpha_k H_k \nabla F(\theta_k),$$

where $H_k$ is a certain matrix that approximates $\nabla^2 F(\theta_k)^{-1}$.

# Gauss-Newton Method for Nonlinear Least Squares

Consider a function

$$\min_{\theta \in \mathbb{R}^P} F(\theta) \triangleq \frac{1}{n} \sum_i F_i(\theta), \text{ where } F_i(w) = f_i(\theta)^2.$$

$$\nabla F(\theta) = \frac{1}{n} \sum_{i=1}^{n} \underbrace{f_i(\theta)}_{1\times 1} \underbrace{\nabla f_i(\theta)}_{P\times 1}.$$

$$\nabla^2 F(\theta) = \underbrace{\frac{1}{n} \sum_{i=1}^{n} \underbrace{\nabla f_i(\theta)}_{P\times 1} \underbrace{\nabla f_i(\theta)^T}_{1\times P}}_{G_1(\theta)} + \frac{1}{n} \sum_{i=1}^{n} \underbrace{f_i(\theta)}_{1\times 1} \underbrace{\nabla^2 f_i(\theta)}_{P\times P}$$

F: loss function

n: #samples / #tokens

P: #params

i: index of sample

用其中一项，代替整个 Hessian 阵！

**Gauss-Newton method:** $\theta_{r+1} = \theta_r - \alpha_r \left(G_1(\theta_r)\right)^{-1} \nabla F(\theta_r),\ r = 0, 1, \cdots$

# Gauss-Newton Perspective on AdaGrad

GN:

$$\theta_{t+1} = \theta_t - \alpha_t \left( \sum_{i=1}^{n} \nabla F_i(\theta_t) \nabla F_i(\theta_t)^\top \right)^{-1} \left( \sum_{i=1}^{n} \nabla F_i(\theta_t) \right)$$

Root-GN (GN with root inverse)

$$\theta_{t+1} = \theta_t - \alpha_t \left( \sum_{i=1}^{n} \nabla F_i(\theta_t) \nabla F_i(\theta_t)^\top \right)^{-\frac{1}{2}} \left( \sum_{i=1}^{n} \nabla F_i(\theta_t) \right)$$

Per sample grad-> per batch grad

$$\theta_{t+1} = \theta_t - \alpha_t \left( \sum_{\mathcal{B}} \nabla F_{\mathcal{B}}(\theta_t) \nabla F_{\mathcal{B}}(\theta_t)^\top \right)^{-\frac{1}{2}} \left( \sum_{\mathcal{B}} \nabla F_{\mathcal{B}}(\theta_t) \right)$$

Here, $F_{\mathcal{B}}(\theta_t) = \frac{1}{B} \sum_{i=1}^{B} F_i(\theta_t)$

Stochastic root-GN:

$$\theta_{t+1} = \theta_t - \alpha_t \left( \sum_{k=0}^{t} g_k g_k^\top \right)^{-\frac{1}{2}} g_k$$

g_k: minibatch grad at k-th step

# Adaptive Gradient Methods

Stochastic root-GN:

$$\theta_{t+1} = \theta_t - \alpha_t \left( \sum_{k=0}^{t} g_k g_k^\top \right)^{-\frac{1}{2}} g_k$$

AdaGrad:

$$\theta_{t+1} = \theta_t - \alpha_t \operatorname{diag}\left( \sum_{k=0}^{t} g_k g_k^\top \right)^{-\frac{1}{2}} g_k = \theta_t - \alpha_t \left( \sum_{k=0}^{t} g_k \circ g_k \right)^{-\frac{1}{2}} g_k$$

$$v_t = v_{t-1} + g_t \circ g_t$$

$$\theta_{t+1} = \theta_t - \alpha_t \frac{g_t}{\sqrt{v_t} + \epsilon}$$

RMSProp (EMA on preconditioner):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t \circ g_t$$

$$\theta_{t+1} = \theta_t - \alpha_t \frac{g_t}{\sqrt{v_t} + \epsilon}$$

Adam (EMA on preconditioner and grad):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t \circ g_t$$

$$\theta_{t+1} = \theta_t - \alpha_t \frac{m_t}{\sqrt{v_t} + \epsilon}$$

# Part 3 Does Adam Converge?

Why Adam was believed to diverge, what actually governs convergence, and the practical advice that follows.

# From SGD to Adam

$$\min_x f(x) := \sum_{i=1}^{n} f_i(x)$$

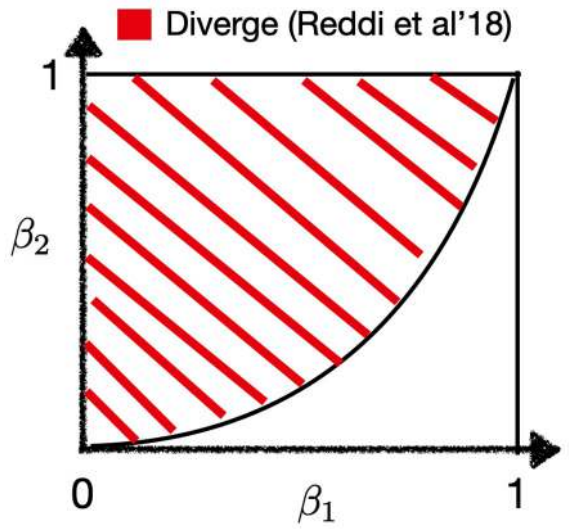
- Adaptive Momentum Estimator (Adam) [Kingma and Ba 15]:
- $m_k = (1-\beta_1)\nabla f_{\tau_k}(x_k) + \beta_1 m_{k-1}$
- $v_k = (1-\beta_2)\nabla f_{\tau_k}(x_k) \circ \nabla f_{\tau_k}(x_k) + \beta_2 v_{k-1}$
- $x_{k+1} = x_k - \eta_k m_k / \sqrt{v_k}$

- **Adam** is the most popular NN training algorithm, default in DL libraries
- $\beta_1$ controls the recursive update of $m_k$; default $\beta_1 = 0.9$
- $\beta_2$ controls the recursive update of $v_k$; default $\beta_2 = 0.999$

# 质疑： Adam 不收敛

Reddi et al.18 **(ICLR Best paper):**

For any $\beta_1, \beta_2$ s.t. $\beta_1 < \sqrt{\beta_2}$ , there exists a problem such that Adam diverges

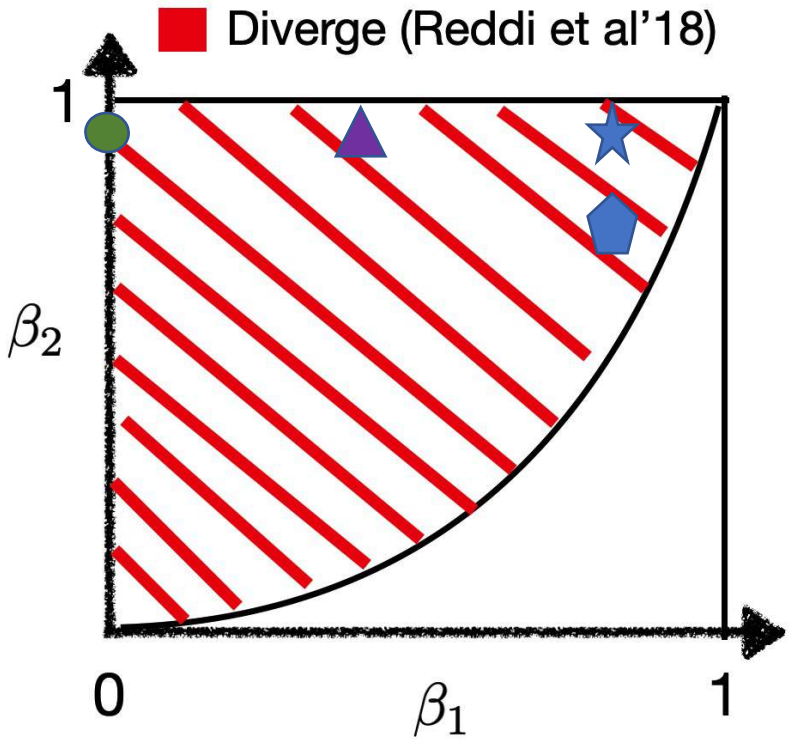



Reddi et al. 开启了长达五年的魔改 Adam 浪潮

Adam 诞生了至少几十个变体 （ AMSGrad, AdaBound, Adashift, … ）

# Back to Adam: Divergence Theory Inconsistent with Practice

**Divergence condition** $\beta_1 < \sqrt{\beta_2}$ (Reddi et al.' 18)




- Most deep learning tasks (e.g. RL, NLP, CV, GAN, etc.) $\beta_1 = 0.9, \beta_2 = 0.999$
- Conditional GAN, DCGAN, etc $\beta_1 = 0.5, \beta_2 = 0.999$
- Super-large language models (e.g. GPT-3) $\beta_1 = 0.9, \beta_2 = 0.95$
- First-order GAN, MSG-GAN $\beta_1 = 0, \beta_2 = 0.999$

# Why is divergence not observed?

- Reddi et al. 18 consider $\min_{\mathbf{x}} f(x) \coloneqq \sum_{i=1}^{n} f_i(x)$

> **Proof**(Reddi et al. 18):
> For any fixed $\beta_1, \beta_2$ s.t. $\beta_1 < \sqrt{\beta_2}$ , we can find an $n$ to construct the divergence example $f(x)$

- An important (but often ignored) feature:
  Reddi et al. fix $\beta_1, \beta_2$ **before picking the problem** (change $n$ according to $\beta_1, \beta_2$ )
- While in practice, parameters are often **problem-dependent**
  (e.g. tuning lr for GD, 我们从来不说GD是发散的算法)

**Conjecture:** Adam might converge for fixed problem (or fixed $n$)

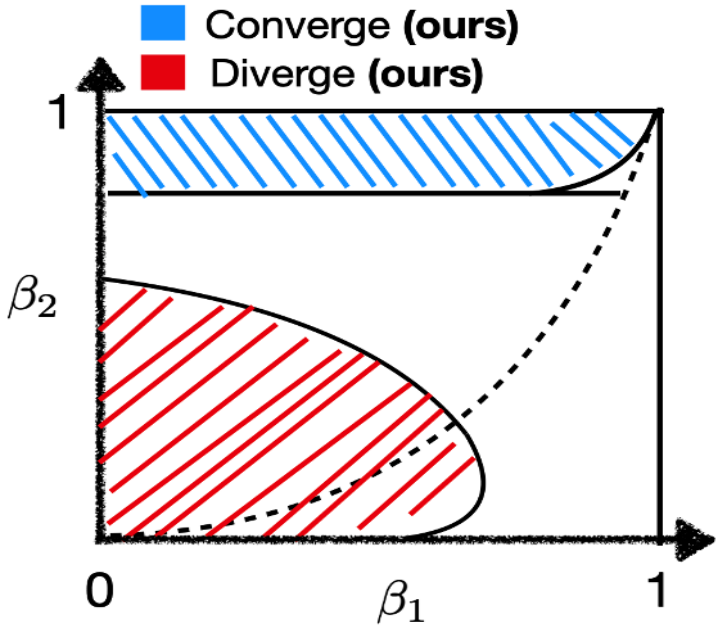
# 2022 年：我们首次证明了 Adam 无需任何修改即可收敛！

**Theorem 1:** when $\beta_2 \geq 1 - O\left(\frac{1-\beta_1^n}{n^{3.5}}\right)$, $\beta_1 < \sqrt{\beta_2} < 1$, Adam converges with rate $O(\frac{\log k}{\sqrt{k}})$

**Theorem 2:** $\exists f(x)$ in the same problem class as above, when $(\beta_1, \beta_2)$ lies in the red region, sequence of $\{x_k\}$ and $\{f(x_k)\}$ of Adam diverges to $\infty$

**Suggestions for hyperparameter-tuning:**

1. Increase $\beta_2$ until convergence
2. Try different $\beta_1 < \sqrt{\beta_2}$ for better performance




Shi, Li, Hong, & **Sun**, RMSprop converges with proper hyper-parameter. ICLR 2021 spotlight.
Zhang, Chen, Shi, **Sun***, & Luo, Adam can converge without any modification on update rules. NeurIPS 2022 spotlight.

# Small beta2 leads to divergence

Consider special case of RMSProp: $\beta_2 = 0$:

$$\text{signSGD: } \theta_{t+1} = \theta_t - \alpha_t \frac{g_t}{\sqrt{g_t \circ g_t}} = \theta_t - \alpha_t \text{sign}(g_t).$$

Consider special case of RMSProp: $\beta_2 = 1$, $v_0 \neq 0$,

$$\text{SGD: } \theta_{t+1} = \theta_t - \alpha_t \frac{g_t}{\sqrt{v_0}}.$$

**Claim 1**: There exists a quadratic problem, such that signSGD (i.e., RMSProp with β2 = 0) with learning rate 1/sqrt(t) diverges.

# Why Small beta2 leads to divergence?

signSGD: $\theta_{t+1} = \theta_t - \alpha_t \text{sign}(g_t)$.

$\nabla F_1(w) = 6w, \nabla F_2(x) = \nabla F_3(w) = -2w.$

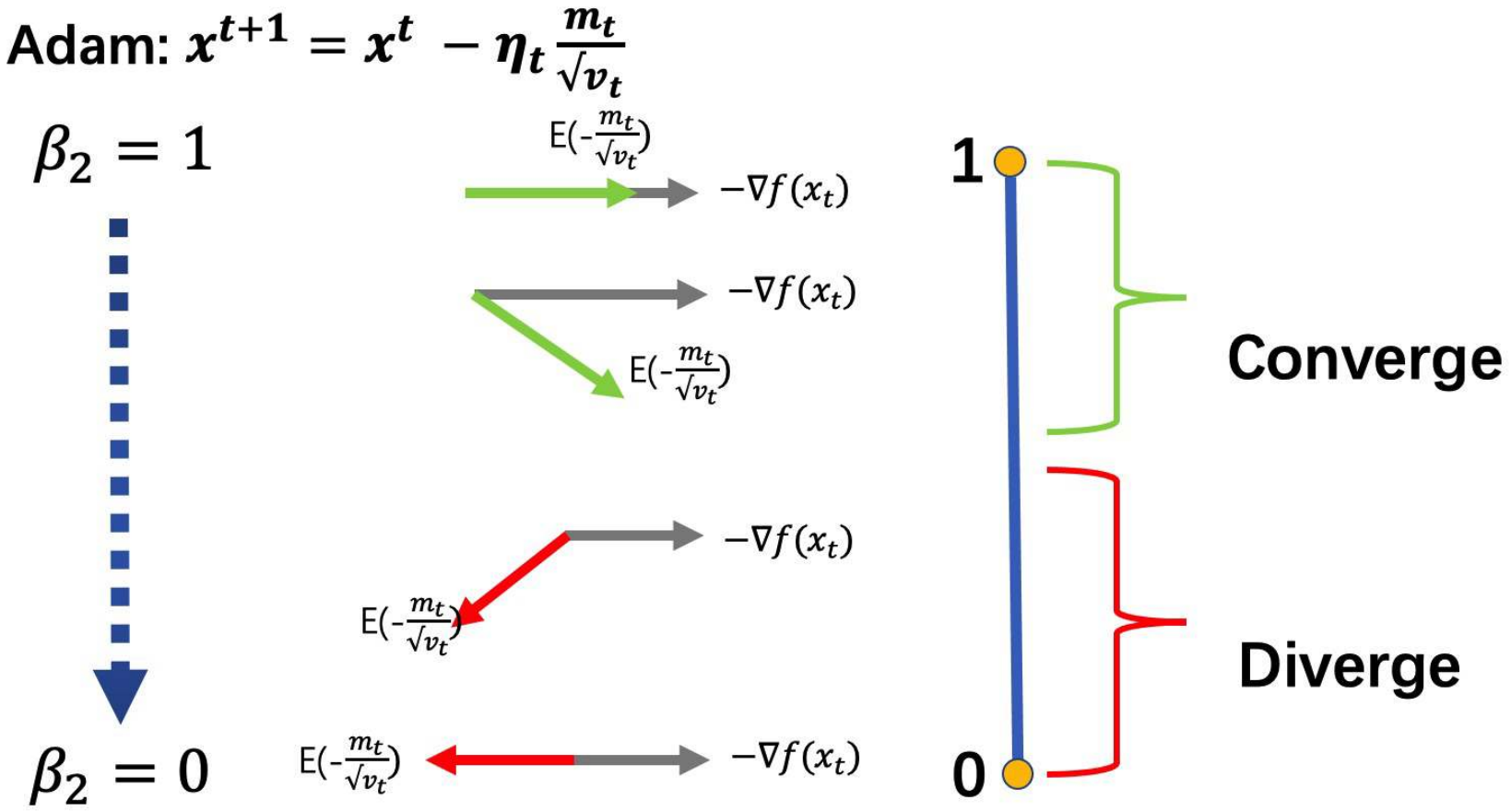
**Expected direction of SGD**:
$E[g_t] = \frac{1}{3}(6w - 2w - 2w) = 2w/3.$

**Expected direction of signSGD**:
$E[\text{sign}(g_t)] = \frac{1}{3}(1 - 1 - 1)\text{sign}(w) = -\frac{1}{3}\text{sign}(w).$

$Angle(E[g_t], E[\text{sign}(g_t)]) = 180^o.$

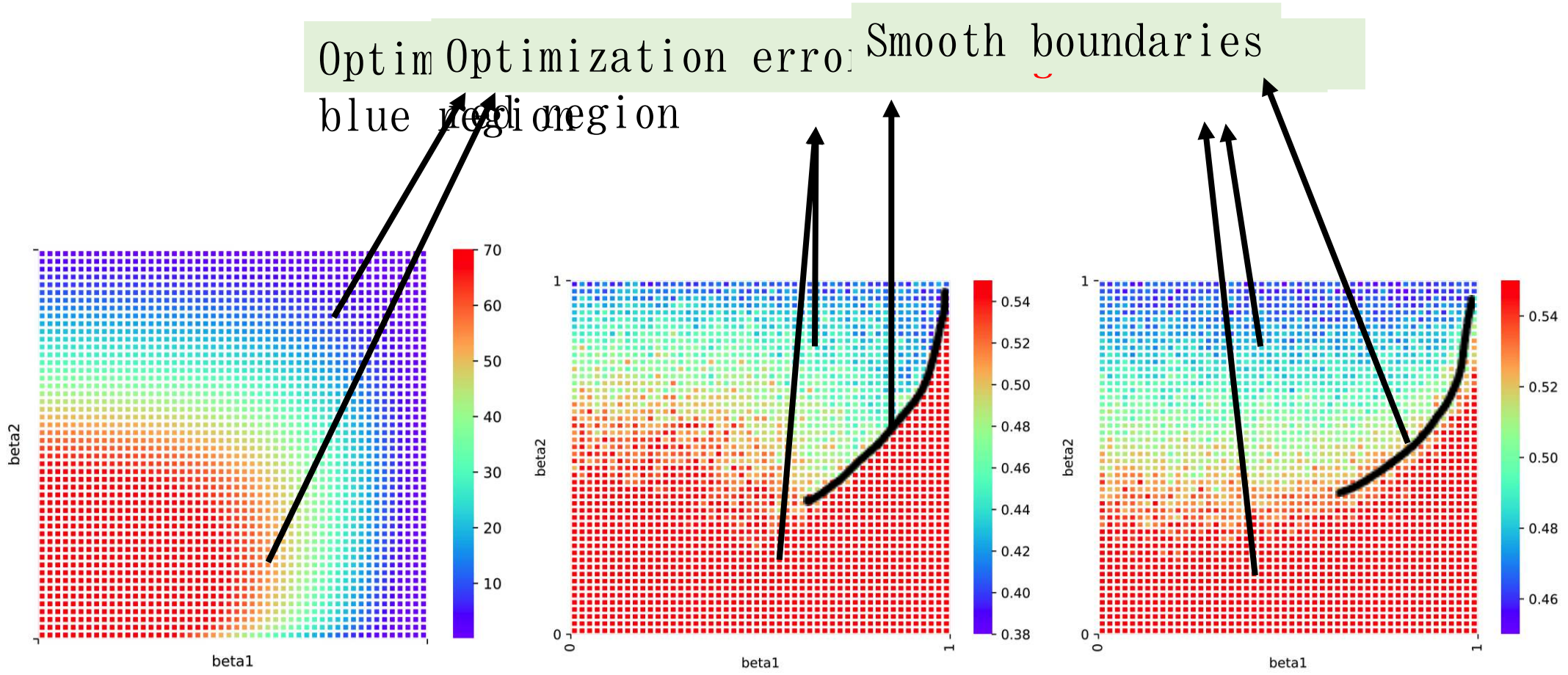
Intuition: Small beta2 makes Adam (RMSProp, signSGD) diverge, because expected update direction can form a obtuse angle with gradient.

# Intuition behind convergence and divergence




**Main difficulties:** characterizing **stochastic non-linear** dynamics

# Our theory is consistent with experiments




(a) Function (2) (b) MNIST (c) CIFAR-10

# Empirical Advice on Tuning Adam

- **Empirical advice:**

  Tune $\beta_2$ larger if performance bad.
  Tune $\beta_2$ larger for larger noise (e.g. smaller batch size)

  Although simple, many practitioners do NOT know this!

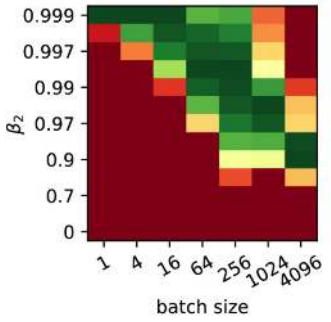
  Setting $\beta_1 = \beta_2$ is close to optimal for almost all neural-nets (they tried).
  [Orvieto, Gower, In Search of Adam's Secrete Sauce, NeurIPS 25, Oral]

# Feedback from LLM developers

**Recent findings by Yair Carmon and coauthors: (on LLM Scaling law)**
" …we find that tuning (enlarging) the $\beta_2$ (from 0.95 to 0.99 and 0.999) is essential at lower batch sizes… This matches the theoretical work by **[Zhang et al. 2022]** that showed that when the batch size is small, higher values of β2 are crucial"

**Recent findings by Harvard & Amazon:**
"larger $\beta_2$ (increase from 0.95 to 0.99, 0.999, or 0.9995) substantially improves small batch size training… aligning with findings in **[Zhang et al., 2022]**"

**Recent findings by Max Planck Institute: "Zhang et al. [2022]** shows that higher $\beta_2$ substantially improve small-batch training, and Marek et al. [2025] highlights the importance of scaling $\beta_2$ in this regime."



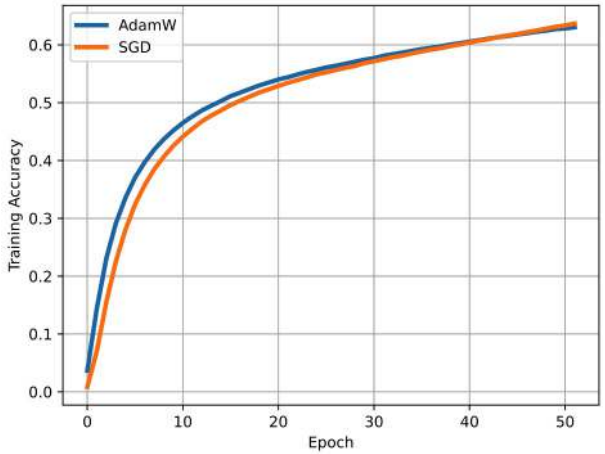

[Figure from Marek et al. 25]

H. Zhang, et al. HOW DOES CRITICAL BATCH SIZE SCALE IN PRETRAINING?, ICLR 2025
Srećković, et al. Is your batch size the problem? revisiting the adam-sgd gap in language modeling, preprint
Porian, T et al., Resolving discrepancies in compute-optimal scaling of language models, NeurIPS 2024

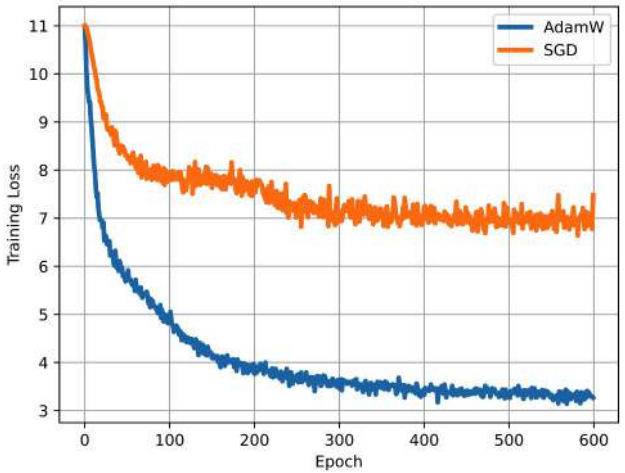
# Part 4 Why Transformers Need Adam

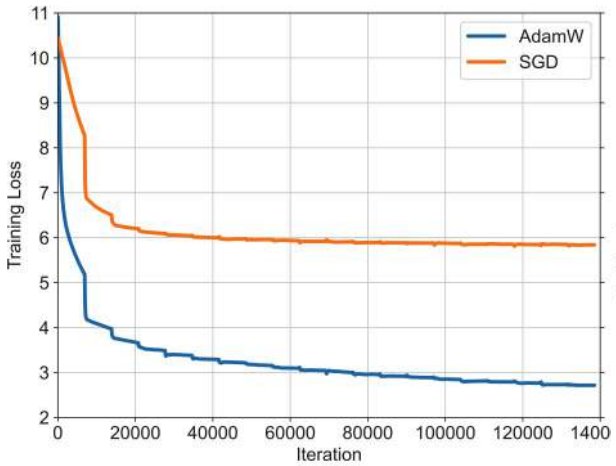
The Hessian of a Transformer is far from isotropic. Block heterogeneity — not the spectrum — is what breaks SGD.

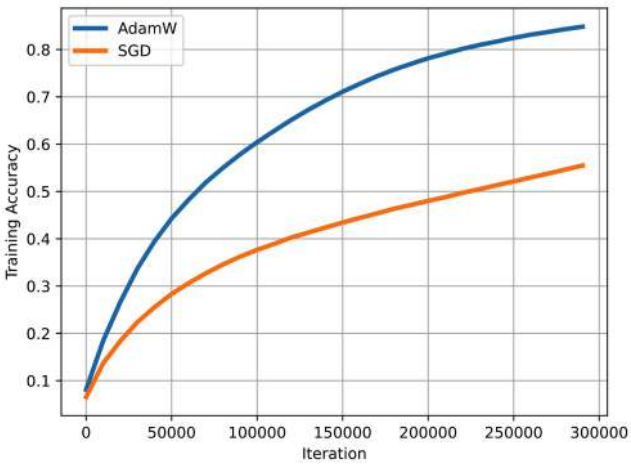
# SGD works well on CNN, largely underperforms Adam on Transformers




VGG (CNN)




GPT2 BERT ViT

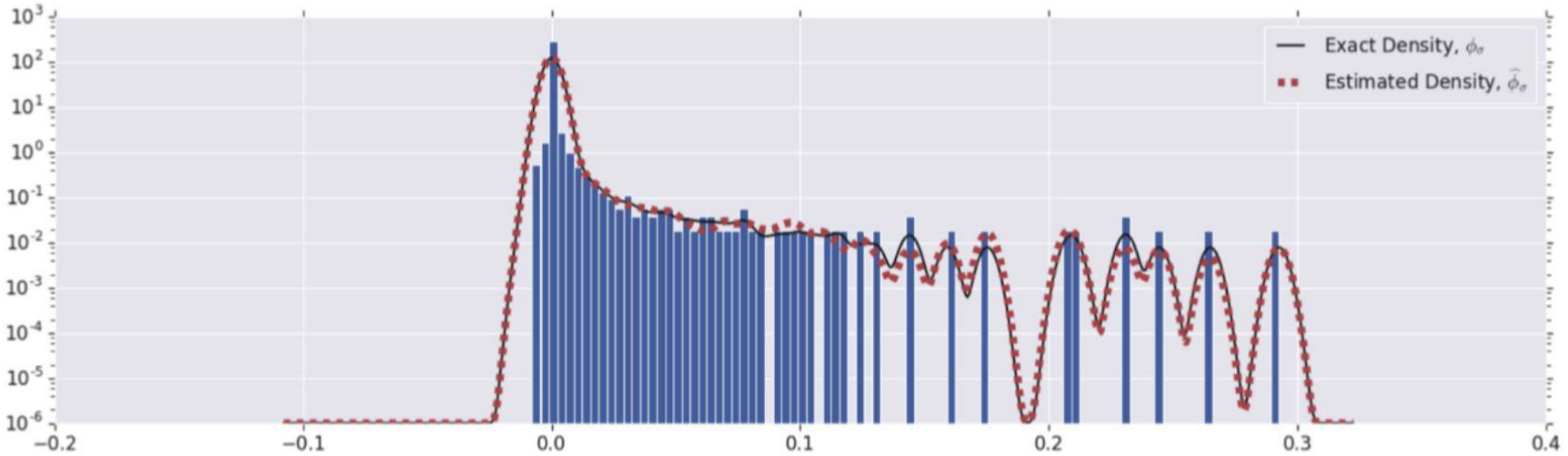
Transformers Need Adam, but why?

# What problem structure might hamper SGD?

- **Hessian eigenvalues** largely decides behavior of gradient methods [Nocedal & Wright'99, Nesterov'13, Goh'17, Sun'2019]
- For instance: ill-conditioning slow down GD
- Can Hessian spectrum explain the gap of SGD and Adam?
- Unfortunately, no (not directly)…

# Preliminary: Hessian spectrum

y-axis: frequency




x-axis: Eigenvalues.

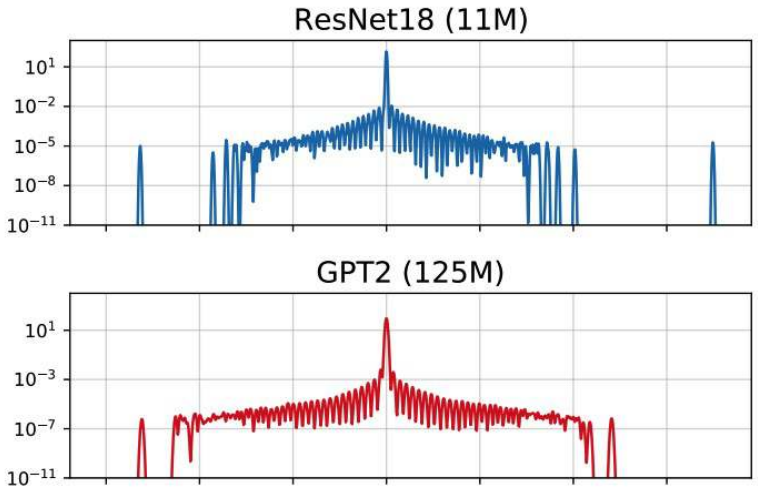
Figure from [Ghorbani et al. 19]

**What is spectrum:** histogram of eigenvalues

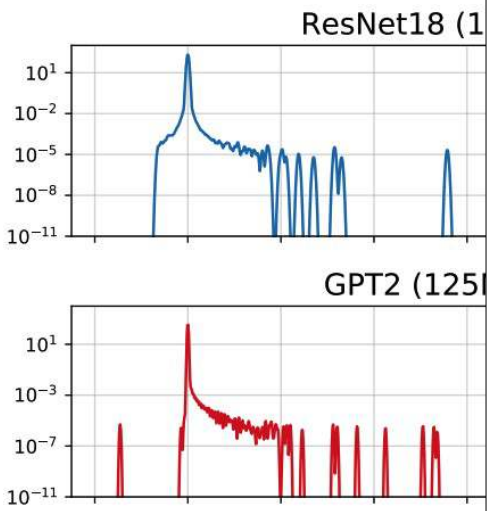
**Remark:** How to plot Hessian spectrum?
We use Stochastic Lanczos Quadrature (SQL)
[Bai, Fahey, and Golub 1996]
(will take >10 pages to explain, omitted today)

We will compute the Hessian spectrum for a wide range of neural networks

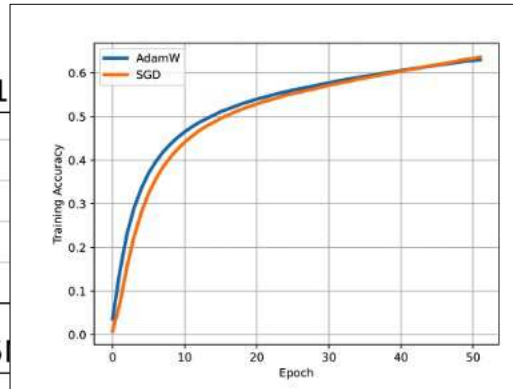
# Hessian spectrum cannot explain the gap



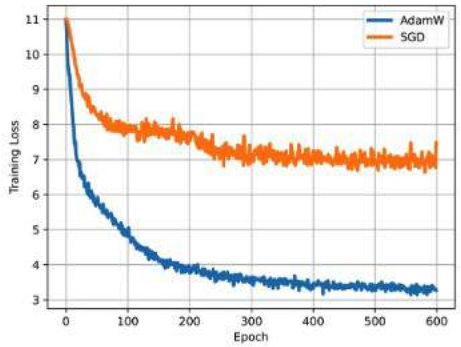

(a) Initialization




(c) 50% ste




SGD ≈ Adam on CNNs



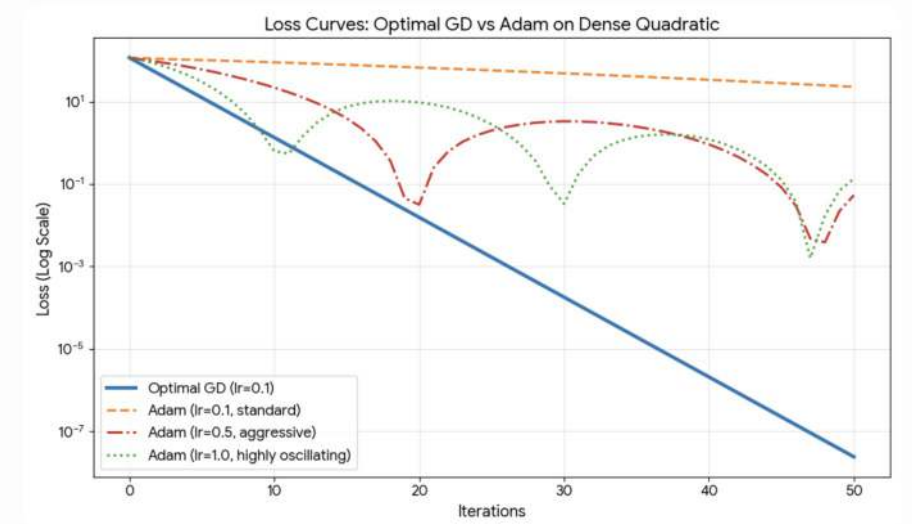

SGD ≪ Adam on Transformers

CNN and Transformers: Spectrum looks quite similar!

(see more figures in the paper)

# GD Beats Adam on Random Quadratic Functions

Even for convex quadratic minimization $min_w\, w^T Aw$, GD can easily beat Adam.
Here is an example, where A is a randomly generated symmetric matrix.




T is seems mysterious:
If Adam cannot beat GD on quadratics, w y can it beat GD on complicated NNs?

NNs must ave special structure t at "random" quadratic problems do not ave

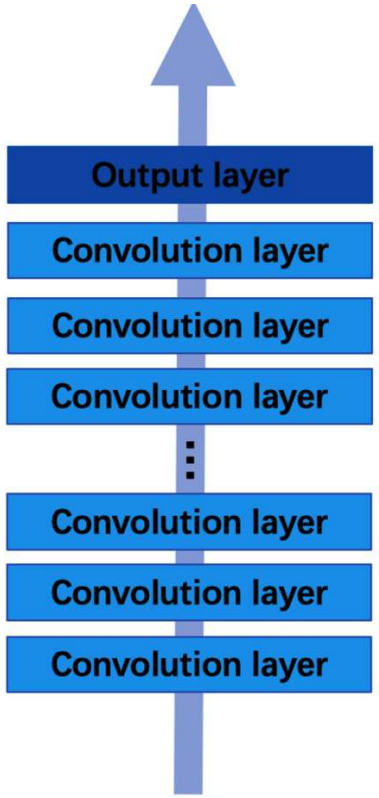
# Something must be overlooked…

- Full hessian spectrum does not seem informative enough
- What else?
- We find one important features that are overlooked:

  The build-up rules of the architecture

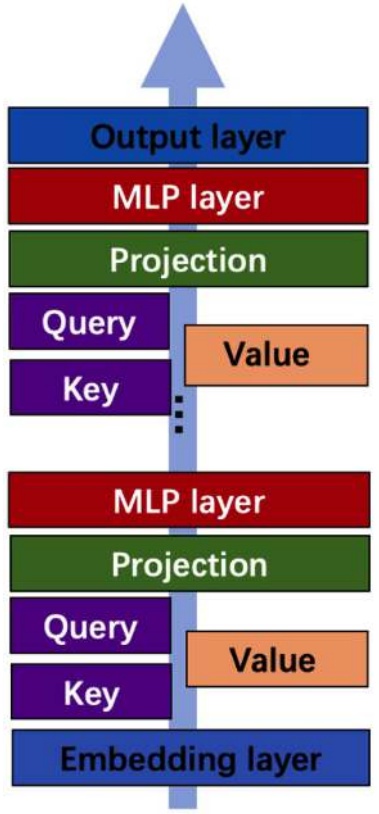
  -- Transformers are stacked up by different kinds of layers

# Build-up rules of architectures




CNNs:

repetitive stack of similar layers

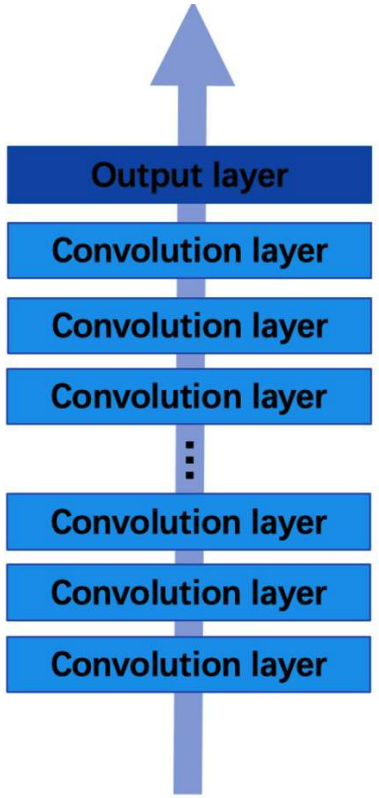



Transformers:

each block follow different design

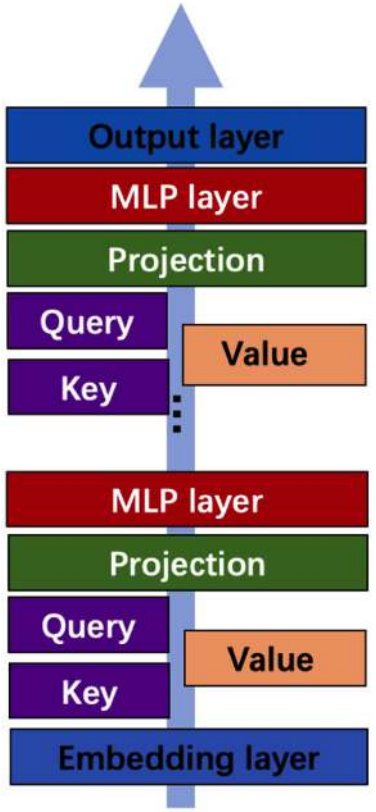
(e.g., Q, K, V, MLP)

# Build-up rules of architectures




CNNs

“周期 =1” 的架构




Transformers

“周期 >1” 的架构

**We hypothesize:**

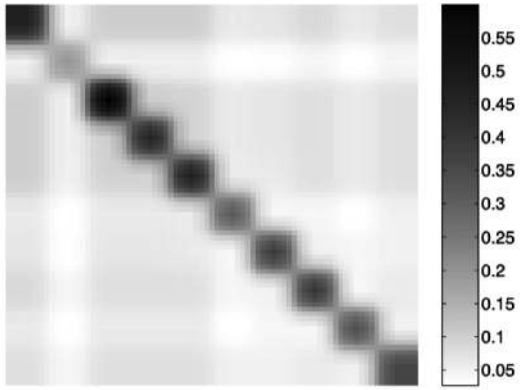
Different designs among parameter blocks

will affect

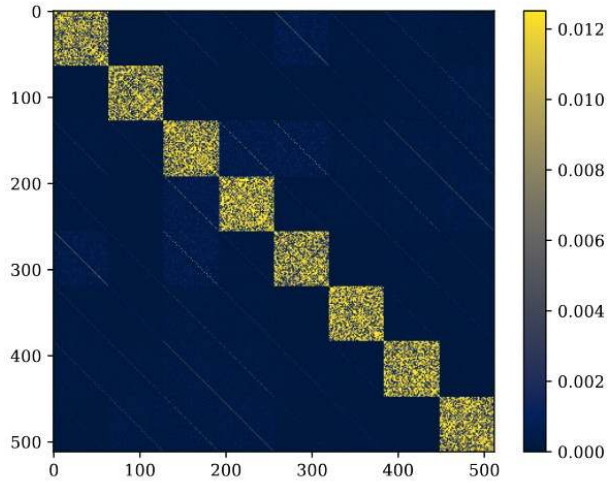
Hessian of these parameter blocks.

This inspires us to investigate the **blockwise Hessian spectra** (i.e., principle diag blocks in Hessian)

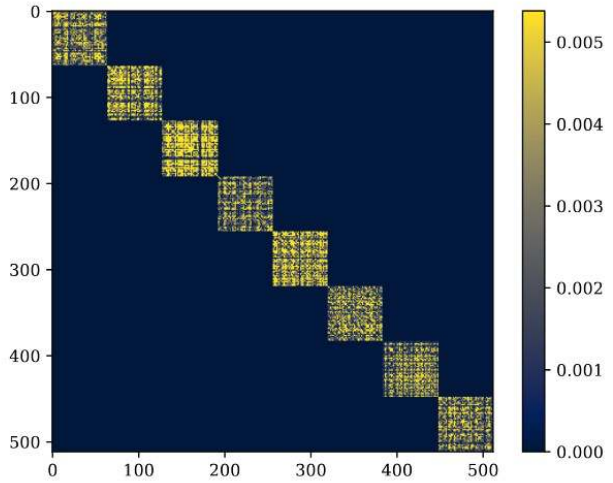
# Another reason for studying blockwise Hessian **near Block Diagonal structure**



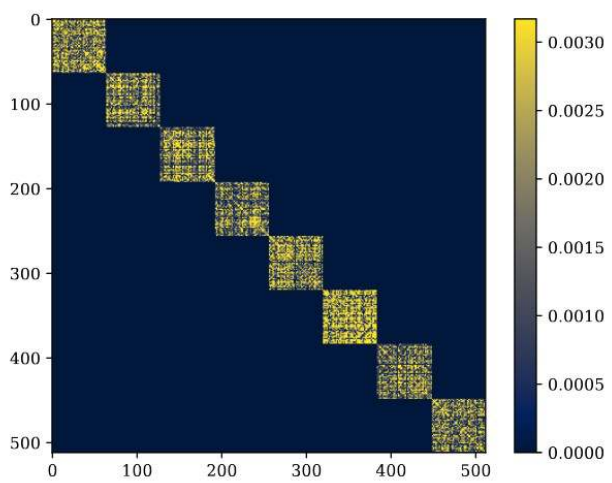

(a) Exact Hessian of a MLP (Collobert, 2004)




(a) Hessian at 5% steps



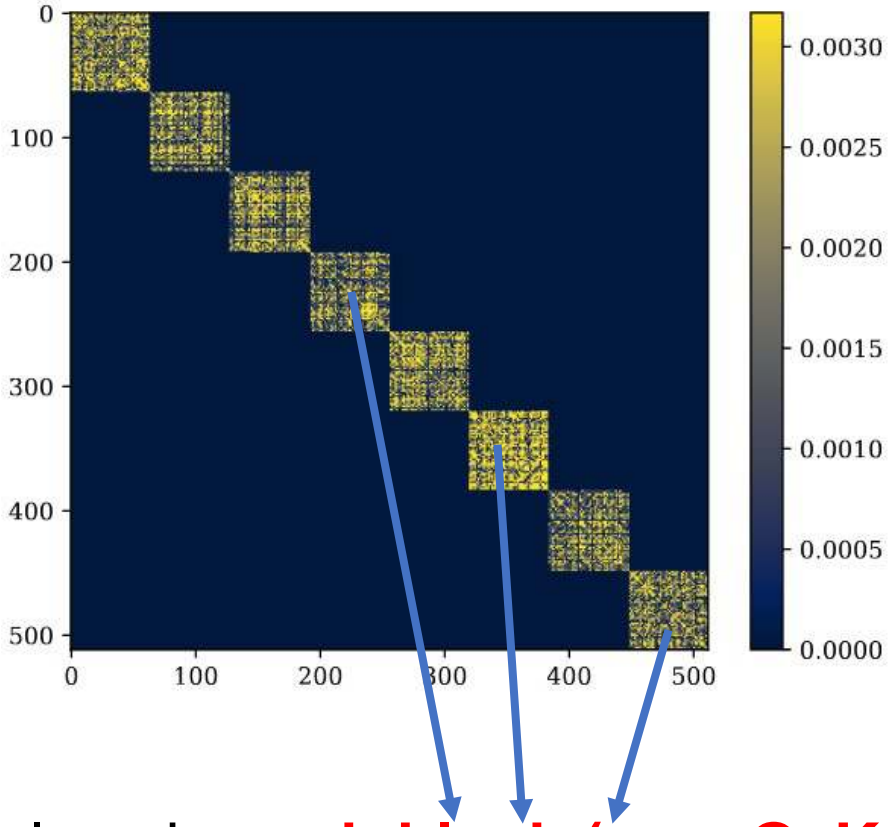

(b) Hessian at 50% steps




(c) Hessian at 100% steps

- 之前没有严谨的理论解释此类 Hessian 结构为什么会出现
- 我们最近用随机矩阵理论给出了第一个严谨的理论解释
- 指出了带来这个结构的关键因素：宽度

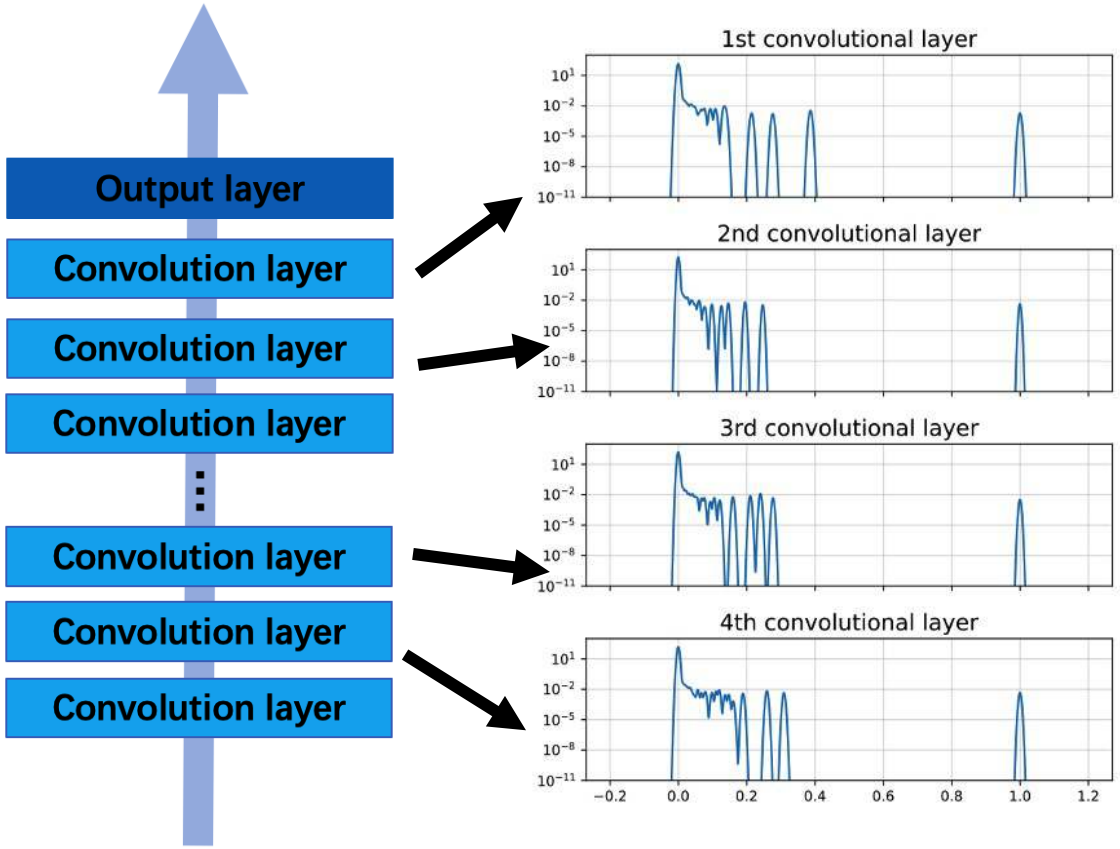
# **Blockwise** Hessian spectrum might matters



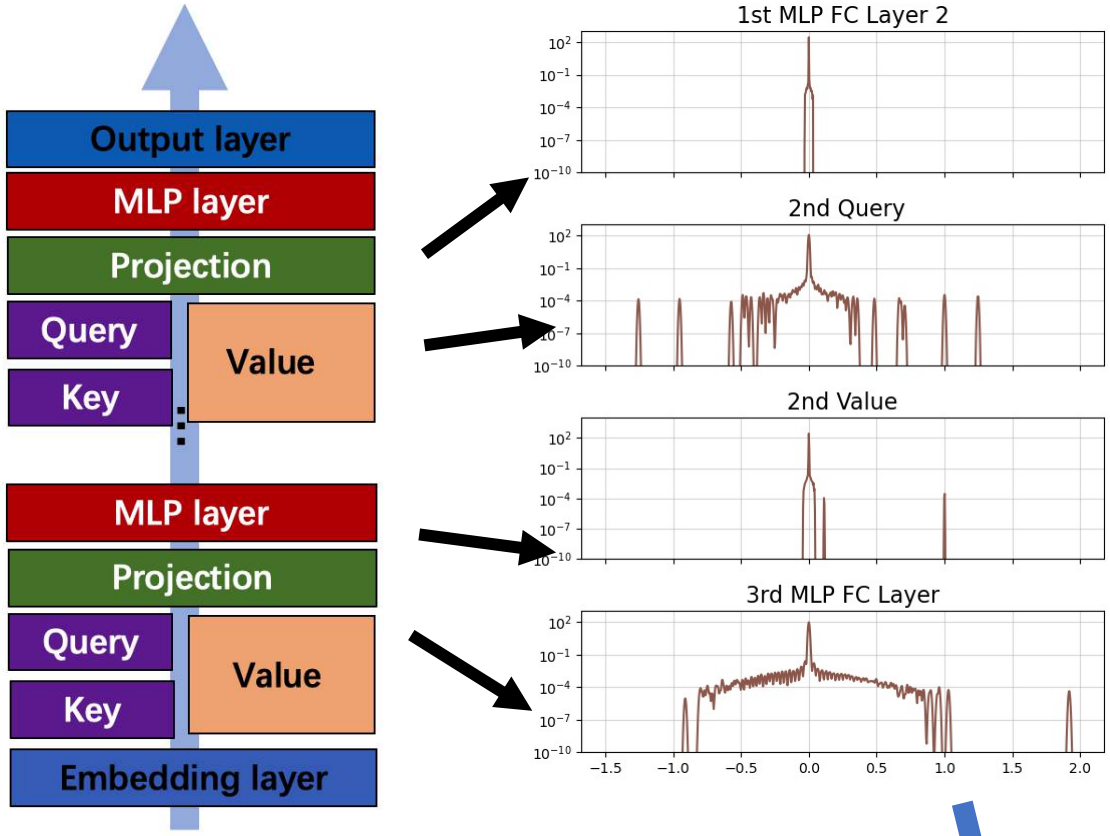

**Conjecture:** Eigenvalues in **each block (e.g., Q, K, V)** could be important

- What extra info over full spectrum?
- By linear algebra: **location** of eigenvalues

# Blockwise Hessian spectrum




ResNet18




BERT

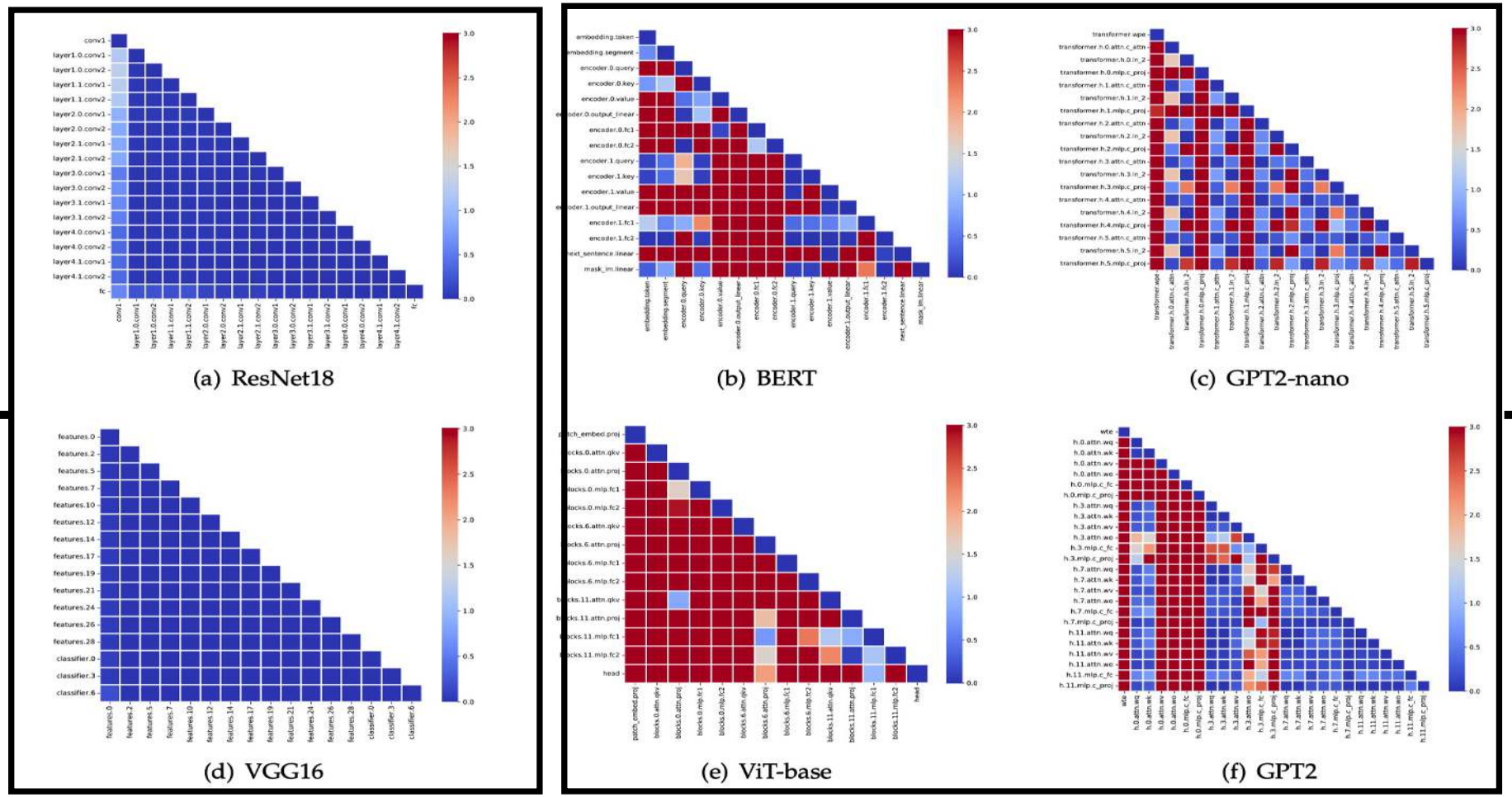
图已 normalize （方便可视化）
实际 Eigen-range 相差 **>200** 倍

CNNs: blockwise spectrum are quite **similar**
We call it ``**homogeneity**”

Transformers: blockwise spectrum are largely **different**
We call it ``**heterogeneity**”

# JS-distance among blocks

CNNs ←




→ Transformers

Figure 4: The JS distance among blockwise Hessian spectra for different models at initialization.

**Observation 1:** Heterogeneity is widely observed in Transformers, but not on CNNs!

# 为什么 Transformer 需要 Adam

**观察 1：**
Transformer 各模块的 Hessian 阵有异质性

**观察 2：**
当模块 Hessian 存在异质性时， SGD 表现差于 Adam

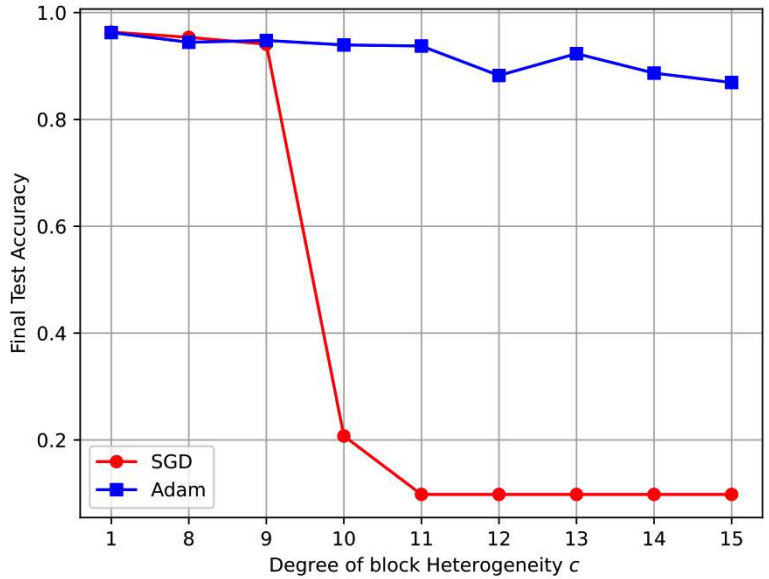
解释

**实验现象：**
训练 Transformer 时，

SGD 效果远不如 Adam

# Hetereogeneity makes SGD worse: Example 2 (pure MLP)

- **Q:** Is Transformer the only architecture that is heterogeneous & SGD worse?
- **A:** No! We provide a few more heterogeneous examples that SGD is worse.




x-axis: degree of Heterogeneity
y-axis: final converged accuracy

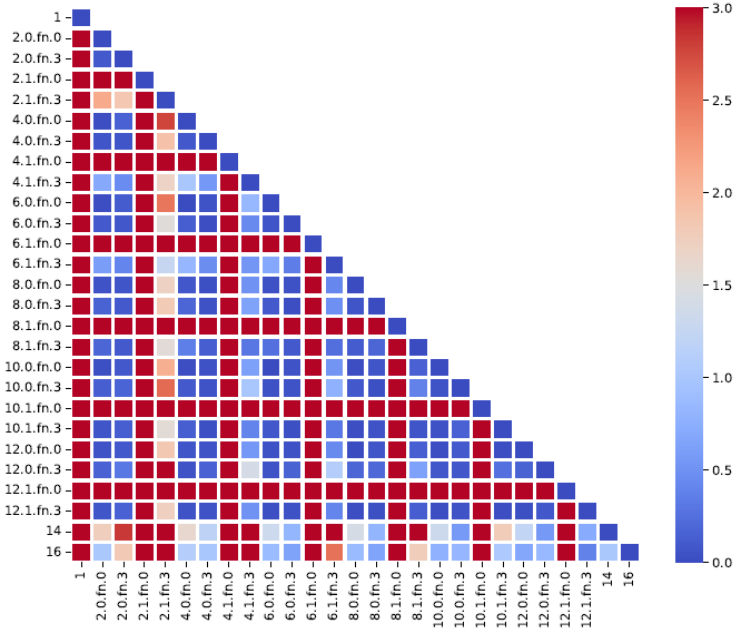
**Example 2:**
A man-made MLP on MNIST:
We exert heterogeneity by scaling each layer differently

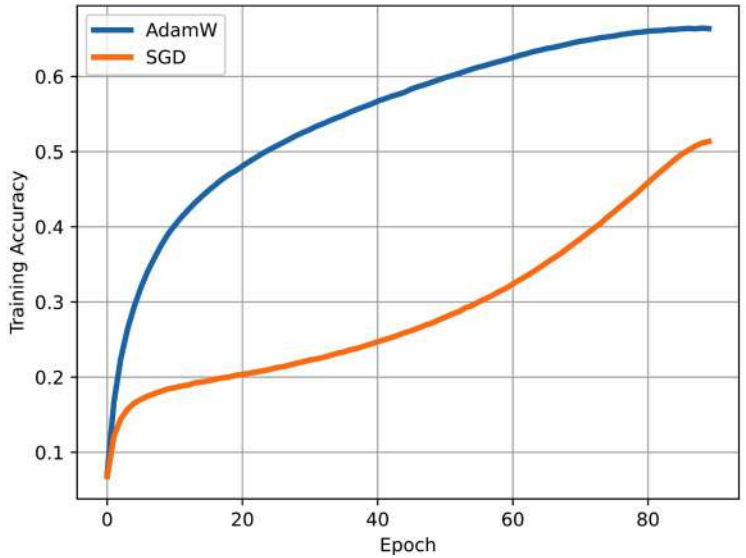
**Observation:**
SGD fails as heterogeneity grows
while Adam remains unaffected

# Hetereogeneity makes SGD worse: Example 3 (MLP-mixer)

**Example 3: MLP-mixer** [1]

A pure MLP architecture that  outperforms CNNs on ImageNet




**Observation:**

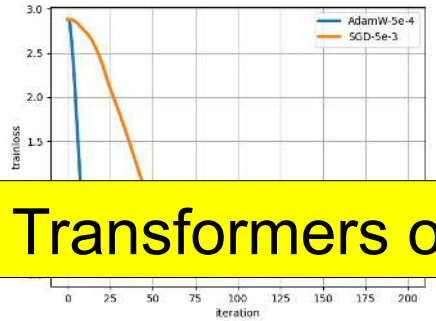
--MLP mixer is heterogeneous

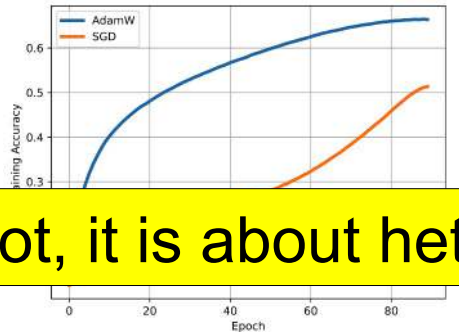
--SGD performs worse than Adam

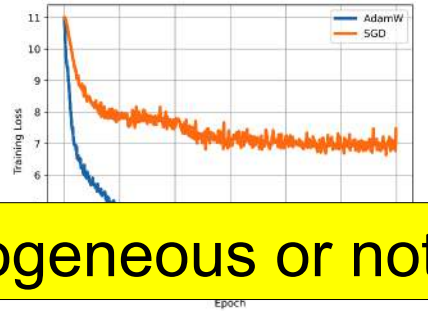
[1] MLP-mixer: An All-MLP Architecture for Vision.  Tolstikhin et al. , NeurIPS 2021

# SGD performance v.s. Heterogeneity in Hessian



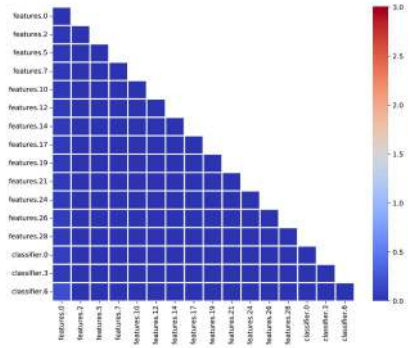

It is NOT about Transformers or not, it is about heterogeneous or not!

Easy for SGD

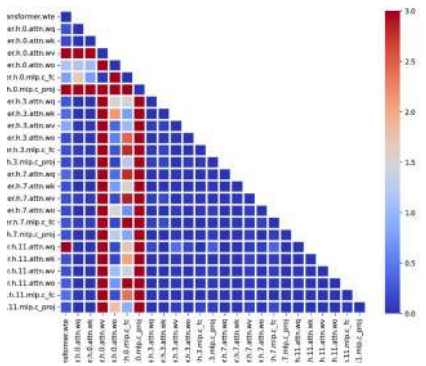
Homogeneity in Hessian

Difficult for SGD

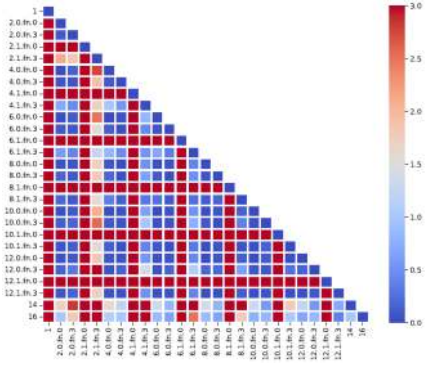
Heterogeneity in Hessian



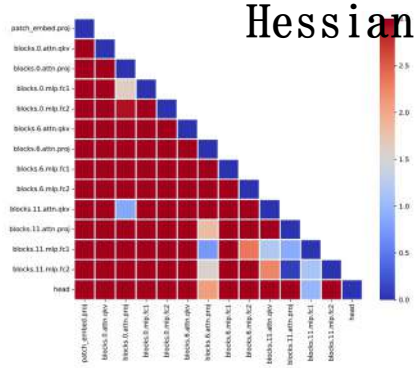
CNN



Transformer (pretrained)



MLP-mixer



Transforme r

# Empirical guidance: choose SGD or Adam?

- Introduce metric to **predict** the failure of SGD **before launching the training**

- **Our metric:** average JS distance of spectrum among blocks at step = 0, called $JS^0$
- This metric could be efficiently computed using Stochastic Lanczos Quadrature
  Our **PyTorch implementation**: https://github.com/zyushun/hessian-spectrum

| Model | ResNet18 | VGG16 | GPT2 (pretrained) | MLP-mixer | BERT | GPT2 | ViT-base |
|---|---|---|---|---|---|---|---|
| $JS^0$ | 0.10 | 0.09 | 18.84 | 34.90 | 53.38 | 83.23 | 286.41 |

For CNNs: 100x smaller than Transformers!

# Theoretical results

**A well-known result for GD**: Consider $\min_w f(w) = \frac{1}{2} w^T H w$ , where $H$ is PD, then there exits a $H$ and $w^0$ :

$$f(w_{GD}^{t+1}) - f^* \geq \left(1 - \frac{2}{\kappa}\right)\left(f(w_{GD}^{t}) - f^*\right)$$

where $\kappa$ is the condition number of $H$

**Theorem 1(Adam):** Consider $\min_w f(w) = \frac{1}{2} w^T H w$ , where $H$ a block-diagonal PD matrix with $L$ blocks, then:

$$f\left(w_{Adam}^{t+1}\right) - f^* \leq \max_{l \in [L]} \left(1 - \frac{1}{\kappa_{Adam,l}}\right)\left(f\left(w_{Adam}^{t+1}\right) - f^*\right)$$

where $\kappa_{Adam,l} = r\,\kappa_l$, $\kappa_l$ is the condition number of $H_l$; $r$ is a constant related to initialization $w^0$

**Remark:** Just special case of Adam wit beta1 = 1;
precise c aracterization of Adam remains largely unexplored.

# Comparing Two Results

- **Compare GD vs Adam:**
  -Adam is faster than GD when $r \max_l \kappa_l \leq \kappa$ ,

Happens in the heterogeneous quadratic examples 1 & 2

Quantity of Adam ~20x smaller, and Adam is also 20x faster

- **This provides a partial theoretical explanation why Adam works better than SGD on Hetergeneous case**

# **Our Explanation:** Why Transformer Needs Adam

**Observation 1:**
Adam is good for block-Hessian.

Part 1

**Observation 2:**
Adam is bad for dense Hessian

Previous slide

Toget er reveals:

**Implication:**
Adam mainly handles "cross-block" challenge,
NOT "within-block" challenge.

**Further Implication:**
We may replace Adam's coordinate-wise lr by block-wise lr

# Part 5 Adam-mini

The same theory turned into practice: one partition rule, one line of code, about half the optimizer memory.

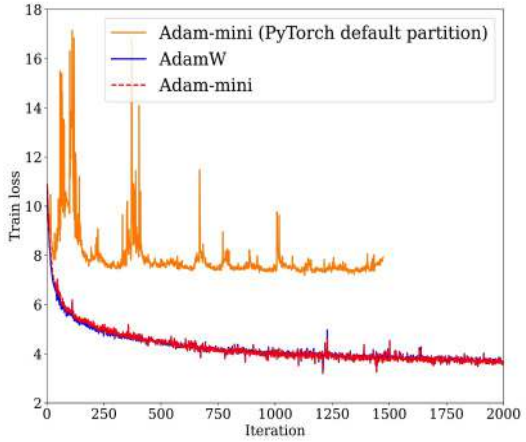
# Adam-mini (Framework)

- **Adam-mini:**
  - **Step 1**: partition the gradient $g$ into B sub-vectors according to the dense Hessian sub-block $g_b, b = [B]$
  - **Step 2:** for each $g_b$, calculate:

$$v_b = (1-\beta_2) * \mathtt{mean}(g_b \circ g_b) + \beta_2 * v_b, \quad b = 1, \cdots B$$

  - **Step 3:** then use $\frac{\eta}{\sqrt{v_b}}$ as the lr for the parameters associated with $g_b$

- **Step 1 is important: need to follow Hessian structure!**
- **Default layer-by-layer partition would fail!**




(i) Llama 2-1B

# Adam-mini: an illustration

- Illustration: For a problem with 5 parameters, $w \leftarrow w - \eta\, u \circ m$

  Adam: $u = (\frac{1}{\sqrt{v_1}}, \frac{1}{\sqrt{v_2}}, \frac{1}{\sqrt{v_3}}, \frac{1}{\sqrt{v_4}}, \frac{1}{\sqrt{v_5}})$

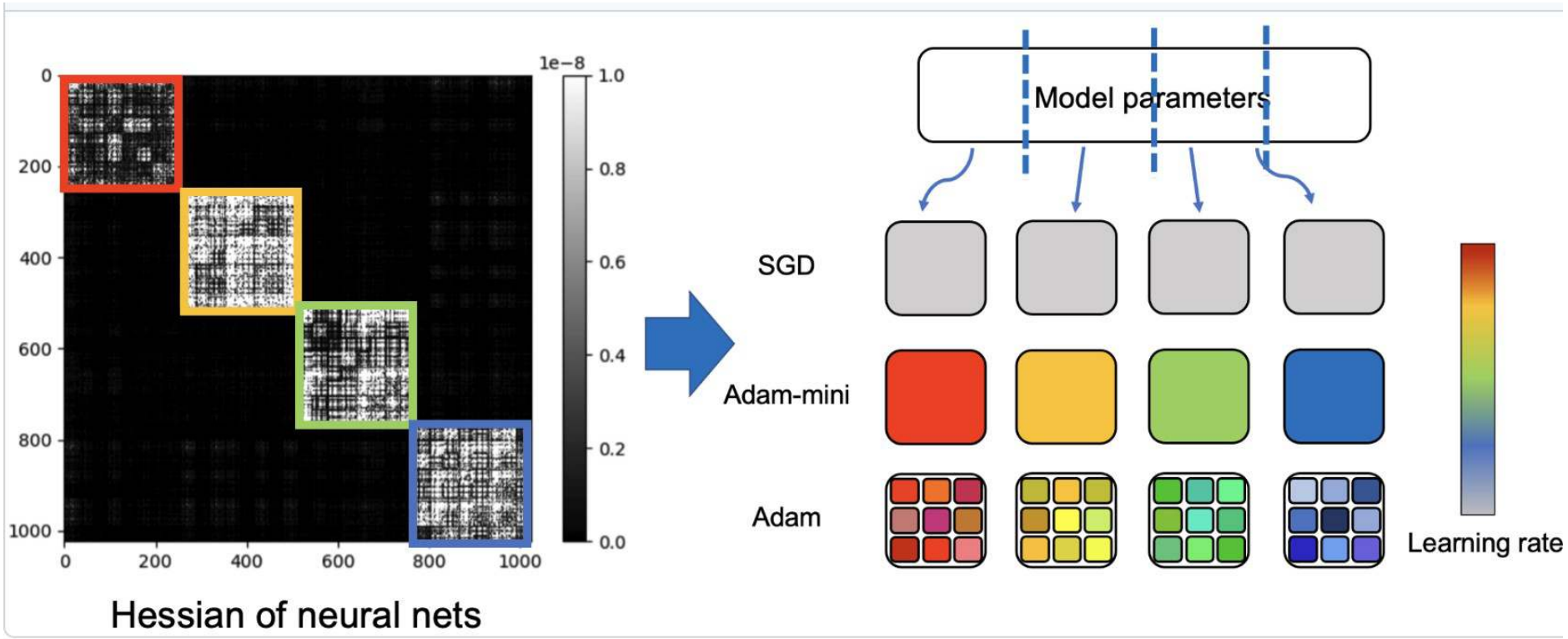
  Adam-mini: if block partition is (1,2,3); (4,5), then

  $$u = (\frac{1}{\sqrt{(v_1+v_2+v_3)/3}}, \frac{1}{\sqrt{(v_1+v_2+v_3)/3}}, \frac{1}{\sqrt{(v_1+v_2+v_3)/3}}, \frac{1}{\sqrt{(v_4+v_5)/2}}, \frac{1}{\sqrt{(v_4+v_5)/2}})$$

- Benefits: reduce # learning rates: from # parameters to # blocks
- For LLMs, we will show that this would free ≥ 90% elements in v
- **Remark:** Cheap way to find "good lrs" , but might not be optimal
- **Remaining question:** How to partition parameters for a given problem, e.g., Transformers?

# Partition Principle

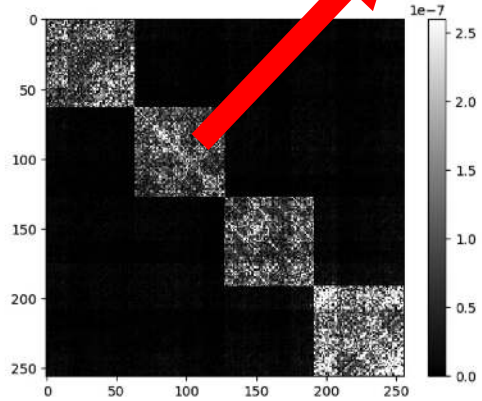
**Partition Principle**:
Partition parameters into blocks s.t. each block is associated with a dense sub-block in Hessian.



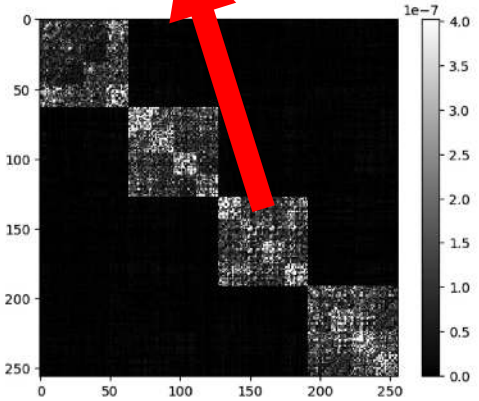
# Applying Principle to Transformer **(non-equal sized blocks)**

#blocks = **#attention heads**

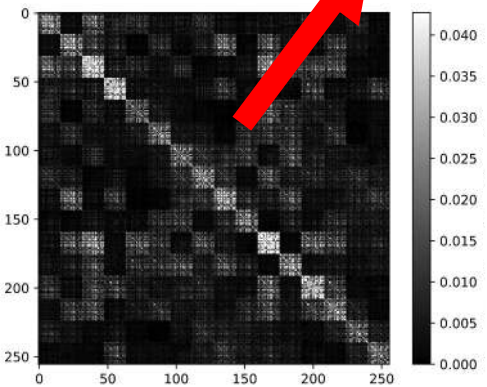
Less clear (treat as **#output neurons**)



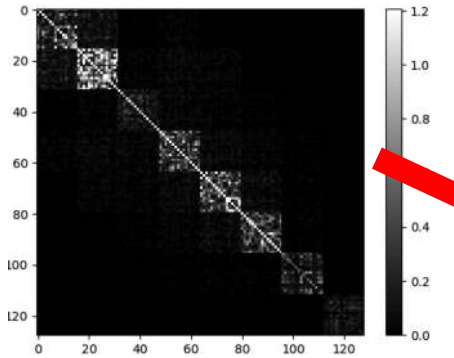
(a) query (4 heads)



(b) key (4 heads)

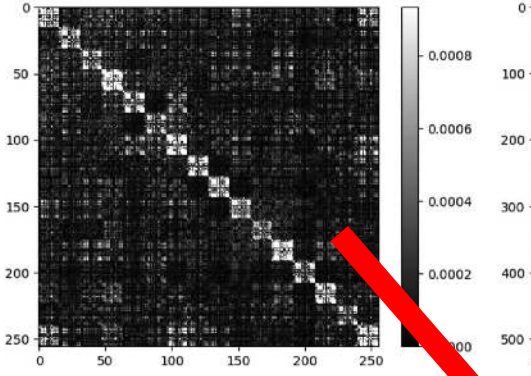


(c) value (4 heads)

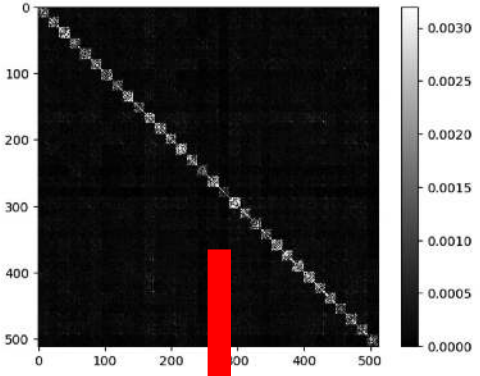


Embedding layer (8 tokens)

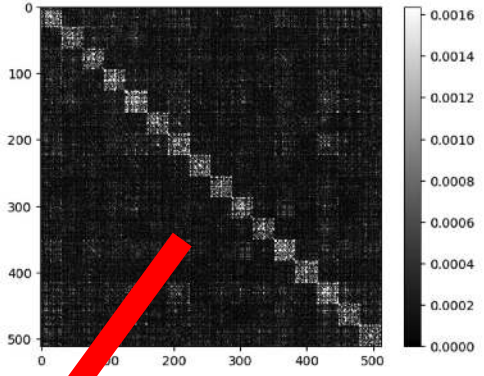
#blocks = **#tokens**



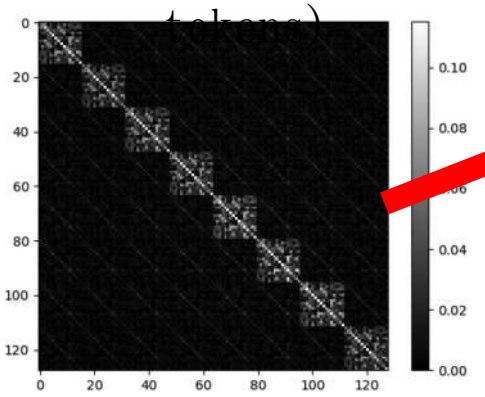
(d) attn.proj (16 neurons)



(e) mlp.fc1 (32 neurons)



(f) mlp.proj (16 neurons)



Output layer (8 tokens)

#blocks = **#output neurons**

# Transformer Partition Strategy

**General Partition Principle**:
Partition parameters into blocks s.t. each block is associated with a dense sub-block in Hessian.

```
Partition Rules for Transformers:

-- For Q & K: each head  is a block.

-- For V, attn.proj, mlp.fc1, mlp.proj: each output neuron is a
block.

-- For embed & output layer: each token is a block
```

# Complete form of Adam-mini

**模块一：**
给定划分方式下的 Adam-mini 算法

**Algorithm 1** Adam-mini in Pytorch style

Input weight-decay coefficient $\lambda$ and current step $t$
Choose `param_blocks` from **Algorithm 2** or **3**
**for** `param` in `param_blocks` **do**
  $g$ = `param.grad`
  `param` = `param` - $\eta_t * \lambda *$ `param`
  $m = (1-\beta_1) * g + \beta_1 * m$
  $\hat{m} = \frac{m}{1-\beta_1^t}$
  $v = (1-\beta_2) * \text{mean}(g \odot g) + \beta_2 * v$
  $\hat{v} = \frac{v}{1-\beta_2^t}$
  `param` = `param` - $\eta_t * \frac{\hat{m}}{\sqrt{\hat{v}}+\epsilon}$
**end for**

模块二：Block 划分方式
Case 1: 非 transformer NN 的 block 划分方式：
按 layers 划分

**Algorithm 2** Partition for non-Transformers

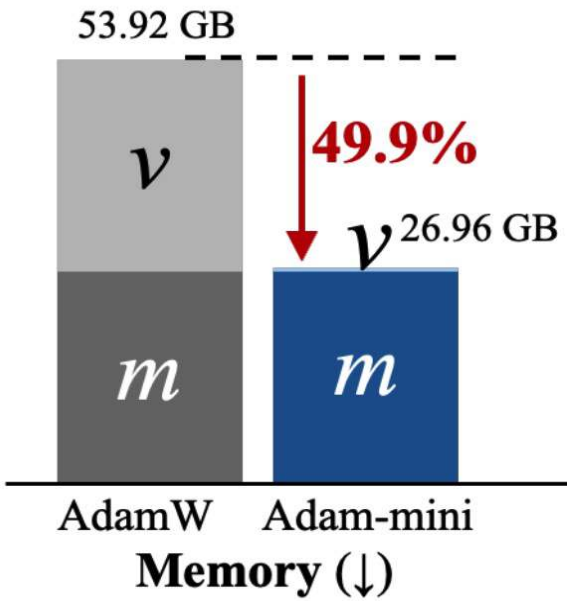
```
param_blocks = {}
for name, param in parameters do
  param_blocks[name]=param
end for
return param_blocks
```

模块二：Block 划分方式
Case 2: Transformer 的 block 划分方式

**Algorithm 3** Partition for Transformers

```
param_blocks = {}
for name, param in parameters do
  if 'embed' or 'output' in name then
    Partition param by tokens
    for i = 0...tokens-1 do
      param_blocks[name+i]=param[i]
    end for
  else if 'query' or 'key' in name then
    Partition param by heads
    for i = 0...heads-1 do
      param_blocks[name+i]=param[i]
    end for
  else if 'value', 'attn.proj', or 'mlp'
  in name then
    Partition param by output neurons
    for i = 0...output_neurons-1 do
      param_blocks[name+i]=param[i]
    end for
  else
    param_blocks[name]=param
  end if
end for
return param_blocks
```

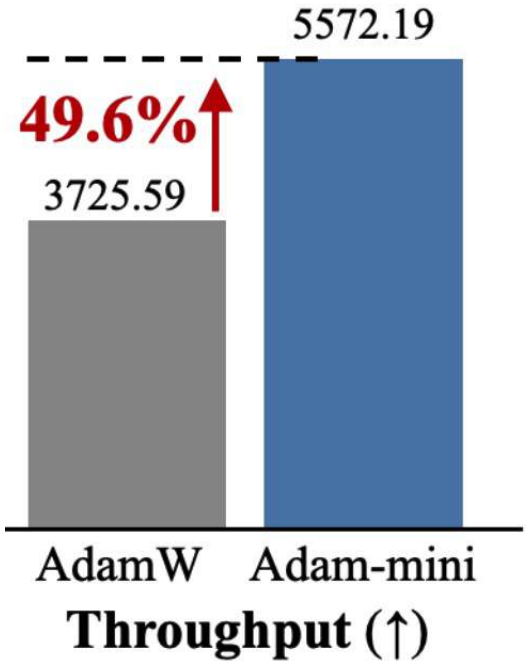
# 内存下降与吞吐量提升




Saves about **50%** memory of Adam

Llama2-7b 中优化器参数数量和内存消耗

| | #m | #v | mem(m) | mem(v) |
|---|---|---|---|---|
| AdamW | 6.74B | 6.74B | 26.95GB | 26.95GB |
| Adam-mini | 6.74B | 1.16M | 26.95GB | 4.65MB |



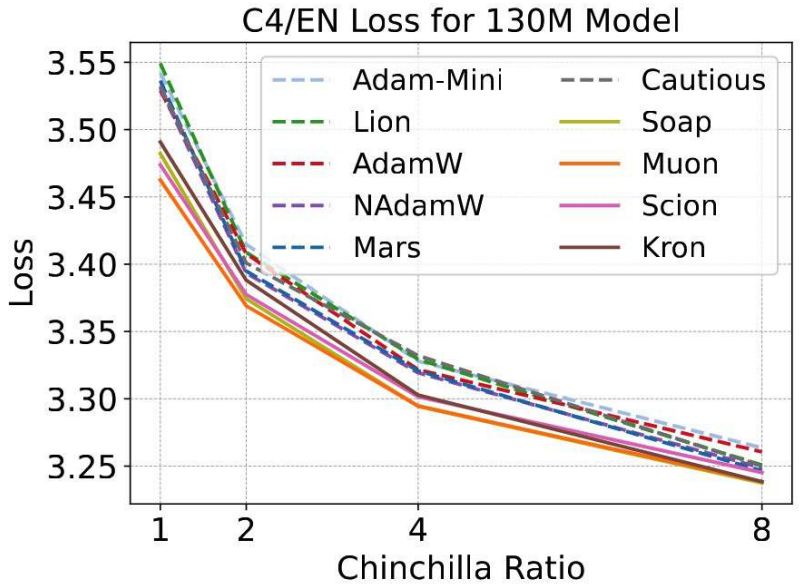

Can increase about **50%** throughput of Adam
(# processed data per second)

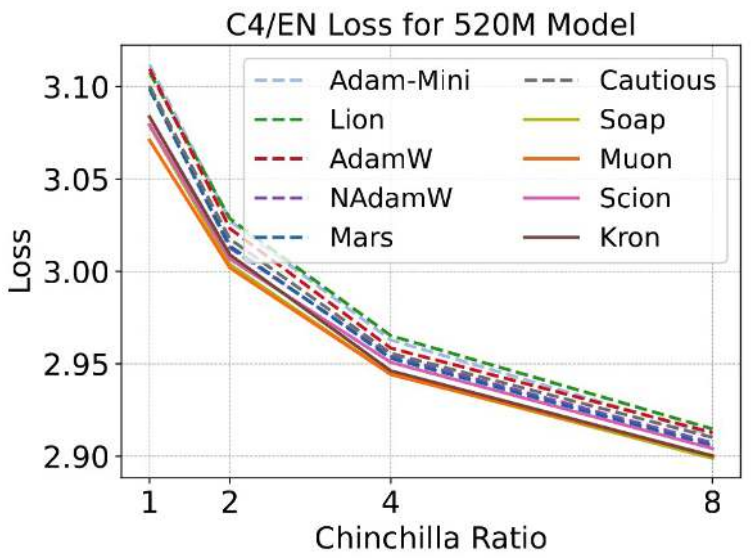
**Why?**

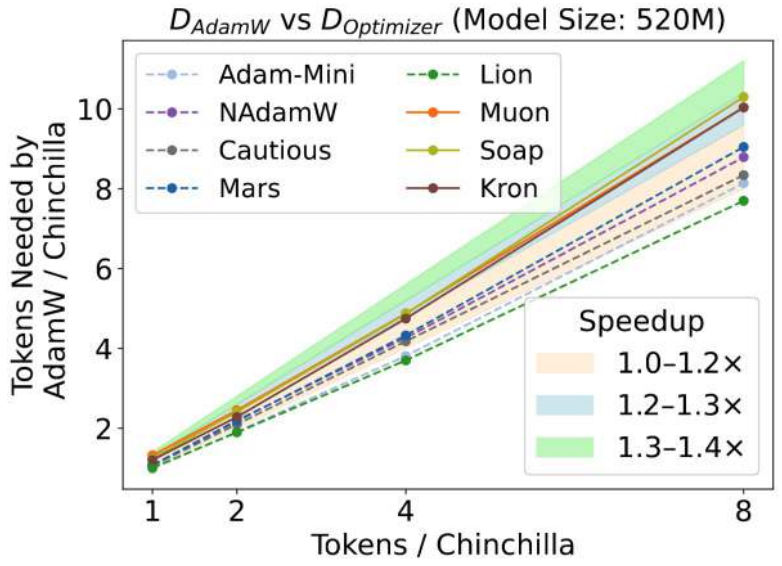
Reduce communication + larger batch size per GPU

# Optimizer Benchmark by Stanford

- 130/ 300/ 520M model with 1x / 2x / 4x / 8x Chinchilla token amount
- Full hyperparameter sweep for all optimizers
- On 520M model, Adam-mini **is slightly faster than** AdamW (though designed




*"Adam-mini closely tracks the performance of AdamW … and sometimes even performs better than AdamW"*

Wen, Hall, Ma, and Liang, (2025). Fantastic pretraining optimizers and where to find them. ICLR 2026.

# How to Use? Just 1-line code change

```
pip install adam-mini
```

```
from adam_mini import Adam_mini
```

```
optimizer = Adam_mini(
            named_parameters = model.named_parameters(),
            lr = lr,
            betas = (beta1,beta2),
            eps = eps,
            weight_decay = weight_decay,
            model_sharding = True,
            dim = model_config.dim,
            n_heads = model_config.n_heads,
            n_kv_heads = model_config.n_kv_heads,
            )
```

Same values as AdamW!

Your model config

**We support:** DDP, FSDP, Deepspeed, Torchtitan, HF trainer 🤗

**Code:** https://github.com/zyushun/Adam-mini



👆 Code for Adam-mini
Currently:
-- 390+ stars
-- 1500+ download via pip install
(in the last two weeks)

If you like Adam, we think Adam-mini is a no-brainer switch!

# Part 6 From Shampoo to Muon

Rising from vectors to matrices: Shampoo, Moonlight, Muon — and the learning-rate and weight-decay rules that make them work.

# Background: Can We Do Better Than Adam?

- **Empirical fact (sad?):** Adam 是长久以来业界 LLM 训练的**唯一选择**

（The only exception: Muon is explored in Moonlight very recently）

能否找到一个算法：

- 效果更好?
- 速度更快?
- 更省内存?

最近，拟二阶方法取得了进展。

# Matrix Preconditioner

Recall: **Adagrad** (and Adam) uses a diagonal reconditioner.

$$g_t = \nabla F_i(\theta_t),$$

$$\theta_{t+1} = \theta_t - \alpha_t \left(\sum_{j=0}^{t} g_j \circ g_j\right)^{-1/2} \circ g_t.$$

It was intended to be an approximation of the GN-method

(technically, it is a variant of GN method, but let's call it GN):

$$\theta_{t+1} = \theta_t - \alpha_t \left(\sum_{j=0} g_j g_j^T\right)^{-\frac{1}{2}} g_t$$

# Matrix-Variable Optimization

- Consider the following matrix-variable finite-sum optimization

$$\min_{W \in \mathbb{R}^{p \times q}} F(W) \triangleq \sum_{i=1}^{n} F_i(W)$$

- If flatten W to w, and let d = pq, then it becomes the problem

$$\min_{w \in \mathbb{R}^d} F(w) \triangleq \sum_{i=1}^{n} F_i(w),$$

  - Mathematically, the same;
  - Computationally, different

Let $G_j$ be the gradient over matrix variable $W_j$, $\dim p \times q$

$g_j = \mathrm{flattened}(G_j)$ , $\dim pq$.

# Time Complexity of **Matrix-Variable Optimization**

Assume **p=q, then** $\mathbf{d} = \boldsymbol{p^2}$.

If flatten the $\mathrm{p}\times q$-matrix variable W to d-dim vector w:

- **GD**: computing gradient takes $O(d^3)$

  --but can be reduced to $O(d^2)$ by parallel computation

- **Naïve Gauss-Newton**: each step $O(d^3) = O(p^6)$

$$\theta_{t+1} = \theta_t - \alpha_t \left( \sum_{j=0}^{t} g_j g_j^T \right)^{-\frac{1}{2}} g_t$$

# Intermediate Method

This leaves a large gap in matrix space!!

**Matrix preconditioned method**:

time of computing inverse $O(p^3)$.

comparable to time of computing gradient $G_t$

$$W_{t+1} = W_t - \alpha_t Q^{-\frac{1}{2}} G_t$$

e.g. let $Q_t = \sum_j G_j G_j^T$

**Remark**: This method is NOT Gauss-Newton.
Sum of $G_j G_j^T$ is NOT the Gauss-Newton matrix.

# Approximating Gauss-Newton Tensor in Matrix-Variable Problem

$G_j$ dim p×$p$ ; $g_j = \text{flattened}(G_j)$ , dim $\boldsymbol{p^2}$.

GN method: $\theta_{t+1} = \theta_t - \alpha_t \left(\sum_{j=0}^{t} g_j g_j^T\right)^{-\frac{1}{2}} g_t$

Should be excellent, But O($p^6$) time

Adagrad $\theta_{t+1} = \theta_t - \alpha_t (\sum_{j=0}^{t} g_j \circ g_j)^{-1/2} \circ g_t.$

O($p^2$) time. Fairly good

Matrix variable expression

Shampoo-v0: $W_{t+1} = W_t - \alpha_t \left(\sum_j G_j G_j^T\right)^{-\frac{1}{2}} G_t$

O($p^3$) time

Shampoo: more principled version, with similar idea…

# Multi-Matrix-Variable Problem

Technically, it is a problem with multiple matrix variables.

Each layer has a matrix $W_l, l = 1,2, \dots, L$.

备选方法一: **Combine all $W_l$ to form a big "matrix" W**
--Challenge: $W_l$'s have different row and column sizes

备选方法二: **Treat $W_l$ separately.**
--Essentially, it is separating Hessian into diagonal blocks!
--Recall: NNs have nearly block diagonal Hessian, so reasonable!

# Shampoo

## Yoram Singer and coauthors in 2018: published Shampoo

Initialize $W_1 = \mathbf{0}_{m\times n}$ ; $L_0 = \epsilon I_m$ ; $R_0 = \epsilon I_n$

**for** $t = 1, \ldots, T$ **do**

Receive loss function $f_t : \mathbb{R}^{m\times n} \mapsto \mathbb{R}$

Compute gradient $G_t = \nabla f_t(W_t)$ $\{G_t \in \mathbb{R}^{m\times n}\}$

Update preconditioners:

$$L_t = L_{t-1} + G_t G_t^\mathsf{T}$$

$$R_t = R_{t-1} + G_t^\mathsf{T} G_t$$

Update parameters:

$$W_{t+1} = W_t - \eta L_t^{-1/4} G_t R_t^{-1/4}$$

$O(p^3)$ time

Algorithm 1: Shampoo, matrix case.

**备注: 这里的 $W_t$, 实际上应该理解为 $W_{l,t}, l = 1, \ldots, L$.**
**对每一层的权重矩阵$W_l$，都采用上面的更新方式.**

Remark:
Matrix power is implemented by using SVD and t en take t e
power of t e singular value.

# Design Idea of Shampoo

**Lemma** Let $g_t = \overline{\mathrm{vec}}(G_t)$

$$\frac{1}{r}\sum_{t=1}^{T} g_t g_t^\top \preceq \left(\sum_{t=1}^{T} G_t G_t^\top\right)^{\frac{1}{2}} \otimes \left(\sum_{t=1}^{T} G_t^\top G_t\right)^{\frac{1}{2}}$$

**Fact** $(L \otimes R^\intercal)\overline{\mathrm{vec}}(G) = \overline{\mathrm{vec}}(LGR)$

Therefore,

$$\begin{aligned}\left(\sum_{j=0}^{t} g_j g_j^T\right)^{-\frac{1}{2}} g_t &\approx \left[\left(\sum_{t=1}^{T} G_t G_t^\top\right)^{-\frac{1}{4}} \otimes \left(\sum_{t=1}^{T} G_t^\top G_t\right)^{-\frac{1}{4}}\right] g_j \\ &\approx \mathrm{vec}\left(\left(\sum_{t=1}^{T} G_t G_t^\top\right)^{-\frac{1}{4}} G_j \left(\sum_{t=1}^{T} G_t^\top G_t\right)^{-\frac{1}{4}}\right)\end{aligned}$$

# Moonlight




基于 openAI 2024 年的优化器 Muon

第一次用到了 16B 模型上

首个公开的非 Adam 优化器
在 10B 以上模型表现超过 Adam

月之暗面最新技术报告 —— 《Muon is Scalable for LLM Training》！ 推出了一个全新的优化器 Muon，并基于此训练出了 3B/16B 参数的混合专家模型 (MoE)——Moonlight。这个 Moonlight 不是普通的模型，它在训练效率上实现了200% 倍提升，并且一举突破了现有大语言模型的性能边界

MUON IS SCALABLE FOR LLM TRAINING

TECHNICAL REPORT

# 国内大模型：拥抱 muon

- **Success:**
  - Scale up by Moonlight (Feb. 2025) from Kimi (**3B/16B MoE model, 5.7T tokens**).
  - Then applied to foundation model training in industry:

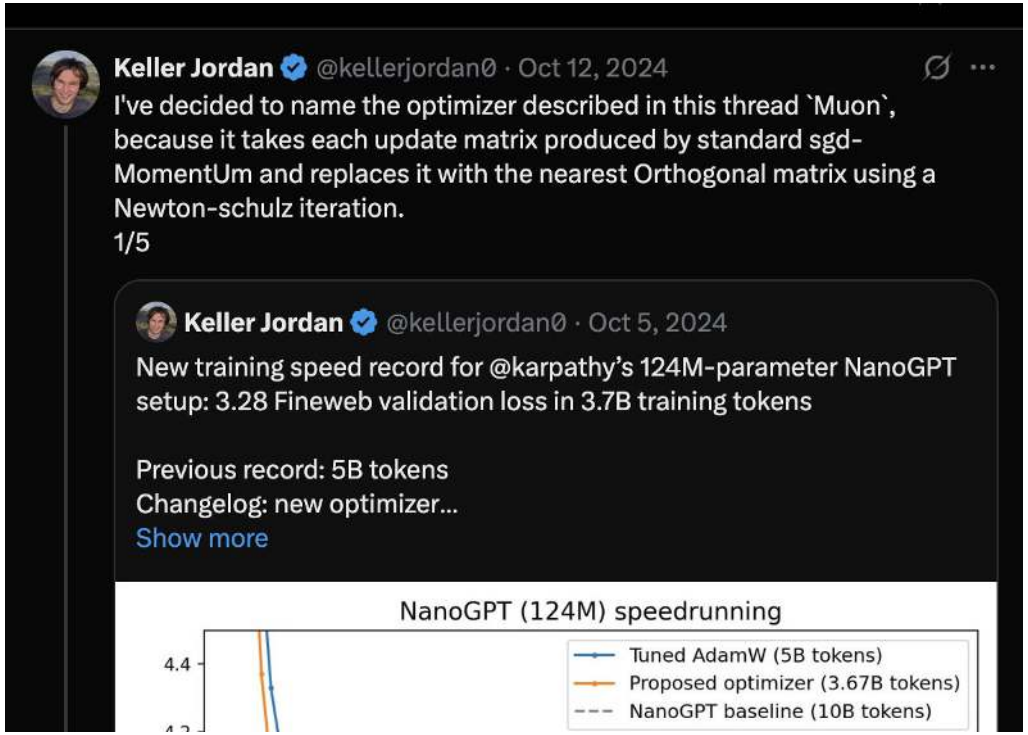
| Model | Date | Model Size (active/total) | Optimizer Note |
|---|---|---|---|
| Kimi K2 | 2025-07 | 32B/1T | MuonClip |
| GLM-4.5 | 2025-08 | 32B/335B | Muon |
| GLM-5 | 2026-02 | 40B/744B | Muon (w/ ead split) |
| Step-3.5-Flas | 2026-02 | 11B/196B | Muon |
| DeepSeek-V4-Pro | 2026-04 | 49B/1.6T | Muon |

Note. Many optimizers have been proposed over the past decade, including Sophia, Lion, and Shampoo, but they have not been validated at very large scale.

**国外大模型用什么优化器？未公开。**
有兴趣的单独讨论。

# Muon 的来源：Twitter 和博客命名

- 该博客是 Kimi Moonlight 文章引用的 Muon 优化器来源
- 但实际上，在 twitter 上，Keller Jordan 已介绍了很长时间




**正式命名为 muon：**

2024 年 10 月 12 日

I've decided to name the optimizer described in this thread `Muon`, because it takes each update matrix produced by standard sgd-MomentUm and replaces it with the nearest Orthogonal matrix using a Newton-schulz iteration.

# Muon 优化器（早期的原始版本）

$$\mathbf{M}_t = \mu \mathbf{M}_{t-1} + \nabla \mathcal{L}_t(\mathbf{W}_{t-1})$$

$$\mathbf{O}_t = \text{Newton-Schulz}(\mathbf{M}_t)^1$$

$$\mathbf{W}_t = \mathbf{W}_{t-1} - \eta_t \mathbf{O}_t$$

**Newton-Schulz Iterations for Matrix Orthogonalization**

$$\mathbf{X}_k = a\mathbf{X}_{k-1} + b(\mathbf{X}_{k-1}\mathbf{X}_{k-1}^{\mathrm{T}})\mathbf{X}_{k-1} + c(\mathbf{X}_{k-1}\mathbf{X}_{k-1}^{\mathrm{T}})^2\mathbf{X}_{k-1}$$

We need to tune t e coefficients so t at t e polynomial f (x) = ax + bx^3 + cx^5 as
a fixed point near 1. In t e original design of K. Jordan et al. 2024, t e coefficients are set to a = 3.4445, b = 4.7750, c = 2.0315 in order to make t e iterative process converge faste

KJ' blog: ttps://kellerjordan.git ub.io/posts/muon

# Note: “Muon” partly uses Adam

- 目前的 Muon，有一部分参数仍然使用 Adam

**Adam 用在**: scalar, vector parameters of the network, input and output layers

**Muon 用在：** 2D and 4D convolutional parameters.

所以 muon 实际上有两个含义：

-- 常见：指代上页的矩阵优化器，

-- 实际运行：指代混合了矩阵优化器和 Adam 的算法

# 第一个修改

- performance gains diminish when we scale up to train a larger model with more token
- 原因： both the weight and the layer output's RMS keep growing to a large scale, exceeding the high-precision range of bf16
- 修改：加 weight decay

$$\mathbf{W}_t = \mathbf{W}_{t-1} - \eta_t(\mathbf{O}_t + \lambda \mathbf{W}_{t-1})$$

Recall: Adam + correct weight decay = AdamW

**其他方法:**

方法一： output 用 Z loss;

方法二：中间层加 weight decay，输入输出不加

# 对比 weight decay 的作用

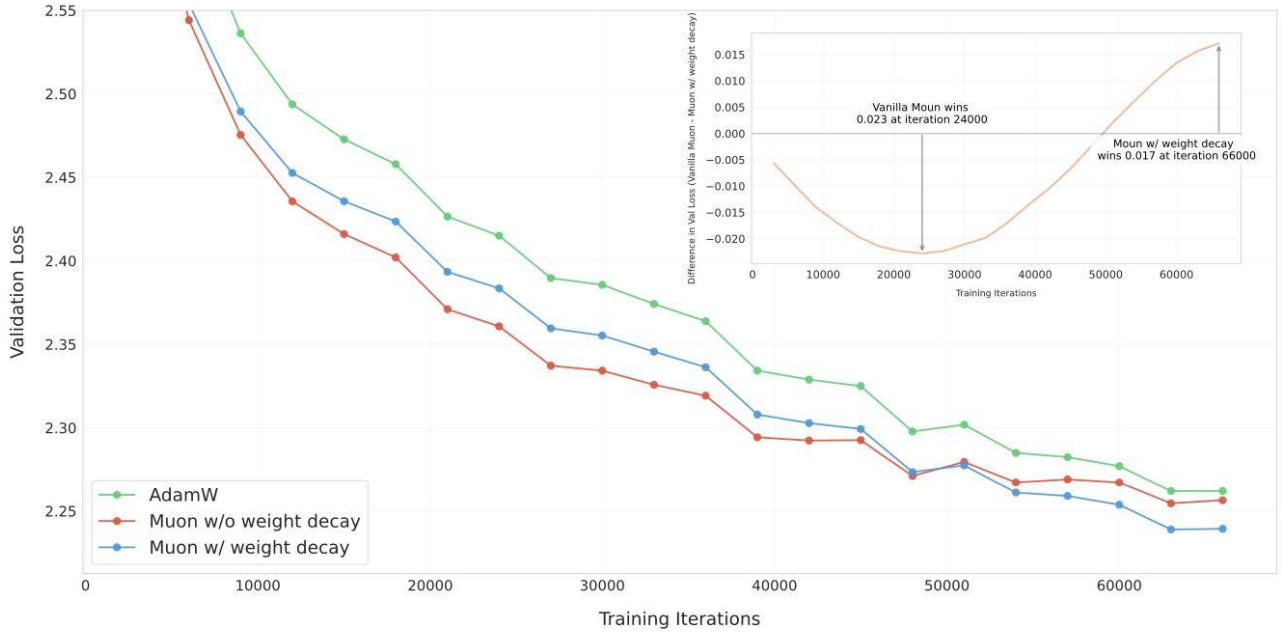



Figure 2: Validation loss curves for AdamW (green), Muon without weight decay (red), and Muon with weight decay (blue).

后期超越 without weight decay 的版本

# 第二个修改：matching RMS

- RMSNorm, LM head, and embedding 用 AdamW
- 希望：超参数 (learning rate η, weight decay λ) 对 matrix and non-matrix parameters 相同

- 观察： AdamW's update RMS is usually around 0.2 to 0.4.
- 修改： scale by 0.2 sqrt{max(A,B)}

$$\mathbf{W}_t = \mathbf{W}_{t-1} - \eta_t(0.2 \cdot \mathbf{O}_t \cdot \sqrt{\max(A, B)} + \lambda \mathbf{W}_{t-1})$$

**分成两大块**： w = (W_1, w_2), update： W_1 用 0.2 eta, w_2 用 eta.

**评论**：实际上就是 2-block 的 blockwise learning rate, similar to Wu Lei's work we mentioned

# Muon optimizer: 更实际的版本

- Weight matrix $W \in \mathbb{R}^{m \times n}$ $(y = Wx; (m, n) = (\text{output dim}, \text{input dim}))$

- **Muon:**

$$G_t = \nabla_W \mathcal{L}_t$$

$$M_t = \beta M_{t-1} + (1 - \beta) G_t$$

$$W_t = W_{t-1} - \eta_t \cdot (\alpha(m, n)\,\mathrm{polar}(M_t) + \lambda W_{t-1})$$

LR adjust factor

Weight decay

- **Orthogonalization:**

$$M = U\Sigma V^\top \longrightarrow \mathrm{polar}(M) = UV^\top.$$

Approximate by Newton-Schultz (NS) iteration

  - **MOTIVATION (Bernstein, original paper):** Steepest descent under spectral norm

# Muon – LR adjust

- $W_t = W_{t-1} - \eta_t \cdot (\alpha(m,n)\text{polar}(M_t) + \lambda W_{t-1})$
- **Moonlight:** $\alpha(m,n) = 0.2\max(m,n)$
  - **Principle:** Match Adam Update RMS
  - **Practice:** Adam part and Muon part **share the same LR.**

- **Keller Jordan:** $\alpha(m,n) = \max(1, \sqrt{n/m})$ ; **muP:** $\alpha(m,n) = \sqrt{n/m}$
  - **muP principle**: **feature update RMS** is O(1).
  - **Practice:** Should set **different LR** for Adam and Muon parts.

**LR suggestion: AdamW part: Muon part = 1:10**

为什么 Adam 的 Update RMS 是 0.2？ ttps://kexue.fm/arc ives/11267
Muon 优化器指南：快速上手与关键细节 ttps://kexue.fm/arc ives/11416
为什么官方版 Muon 比 MuP 版多出一个 max(1, ·)？

# Newton Iteration

- **Newton Iteration** in Higham (2008)'s book, Eq 5.22

**Newton Iteration**

$X_0 = G/\| G \|$ 或 $G/\| G \|_F$

$X_{t+1} = 1.5\, X_t - 0.5\, X_t X_t^T\, X_t.$

**Claim**: 当$\mathrm{t} \to \infty$, $X_t \to UV^T$.

# Newton-Schulz iteration Appendix A of [B&N]

- [B&N] 提出了“ Newton-Schulz iteration”

  发现 [Björck and Bowie, 1971; Kovarik, 1970] 提出过

- 更一般的，可以用如下 iteration 代替 Newton iteration

$$\boldsymbol{X}_{t+1} = a \cdot \boldsymbol{X}_t + b \cdot \boldsymbol{X}_t \boldsymbol{X}_t^\top \boldsymbol{X}_t + c \cdot (\boldsymbol{X}_t \boldsymbol{X}_t^\top)^2 \boldsymbol{X}_t + \ldots + z \cdot (\boldsymbol{X}_t \boldsymbol{X}_t^\top)^n \boldsymbol{X}_t$$

- 选此多项式的原因，以及系数的选择方法：

$$g(x) = \sum_{k=0}^{n} a_k x^{2k+1} = a_0 x + a_1 x^3 + \cdots + a_n x^{2n+1}$$

能有效逼近符号函数 sign(x).

# From Shampoo to Muon

Shampoo (2023)
改进版

$$L_t = \beta_2 L_{t-1} + (1-\beta_2) G_t G_t^\top$$
$$R_t = \beta_2 R_{t-1} + (1-\beta_2) G_t^\top G_t$$
$$M_t = \beta_1 M_{t-1} + (1-\beta_1) G_t$$

$$W_{t+1} = W_t - \eta L_t^{-\frac{1}{4}} M_t R_t^{-\frac{1}{4}}$$

简化为单边 Shampoo

$$L_t = \beta_2 L_{t-1} + (1-\beta_2) G_t G_t^\top$$
$$M_t = \beta_1 M_{t-1} + (1-\beta_1) G_t$$

$$W_{t+1} = W_t - \eta L_t^{-\frac{1}{2}} M_t$$

Update 展开写成

$$W_{t+1} = W_t - \eta \left( \sum_{k=0}^{t} \beta_2^{t-k} G_k G_k^\top \right)^{-\frac{1}{2}} \left( \sum_{k=0}^{t} \beta_1^{t-k} G_k \right)$$

Preconditioner:
EMA of square -> square of EMA

$$W_{t+1} = W_t - \eta \left[ \left( \sum_{k=0}^{t} \beta_2^{t-k} G_k \right) \left( \sum_{k=0}^{t} \beta_2^{t-k} G_k \right)^\top \right]^{-\frac{1}{2}} \left( \sum_{k=0}^{t} \beta_1^{t-k} G_k \right)$$

Muon: let beta1 = beta2

$$W_{t+1} = W_t - \eta (M_t M_t^\top)^{-\frac{1}{2}} M_t$$

# 从 Adam 到 Shampoo 到 muon

- **Adam**: 是 GN 方法的近似，用对角阵近似整个 GN 矩阵

- **Shampoo**: 是 GN 方法的更好的近似，考虑矩阵变量，用分块对角阵来近似整个 GN 矩阵

  -- 和 Adam 的关系：从向量空间升到矩阵空间；其他类似

- **muon**: 是 GN 方法的另一种近似

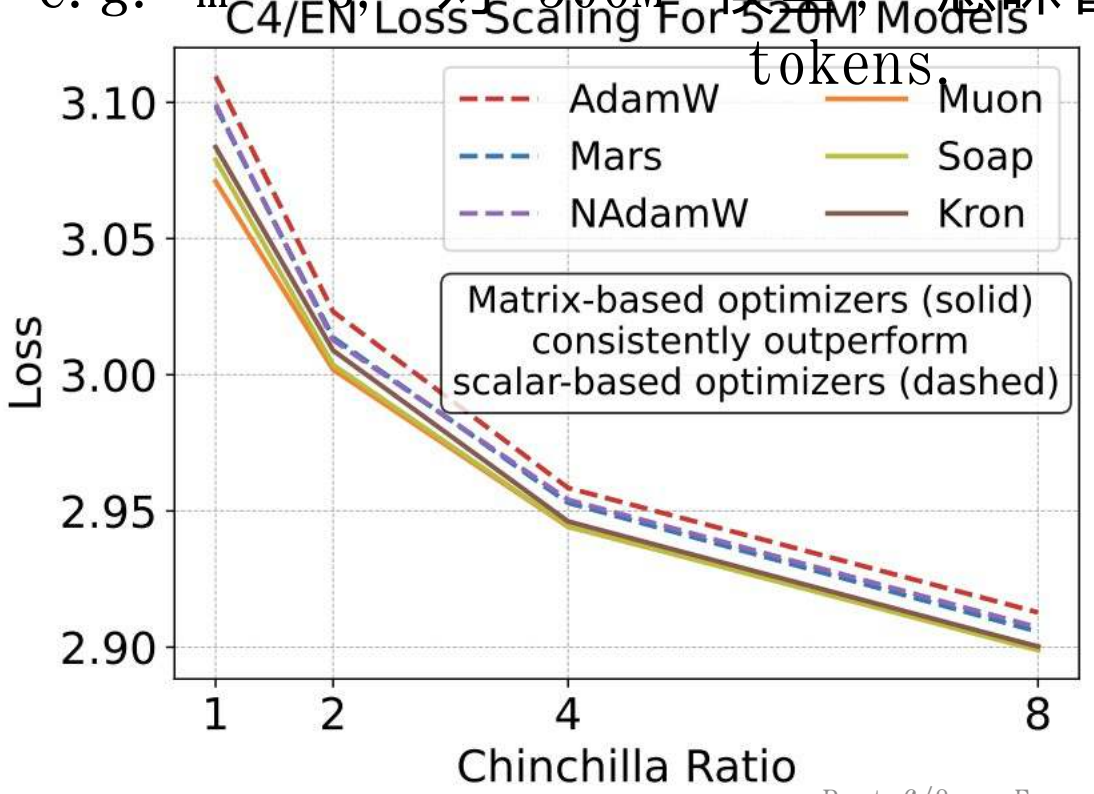
  -- 和 Shampoo 的关系：

  在 Shampoo 中用一阶统计量来估计二阶统计量，即可得到 muon

# Stanford Benchmark: Token 数量增加的影响

Chichilla Ratio = m, 意味着使用 m 倍 Chichilla optimal tokens.

e.g. m = 8, 对 500M 模型, 意味着训练 500*20*8 = 80B tokens.




**矩阵算法好于 scalar 算法。**

**矩阵算法的对比：**

10B tokens:

muon 显著好于 Soap 和 Kron.

80B tokens:

muon 和 Soap, Kron 几乎一样。

# Part 7 Warmup and Schedules

Two things the update rule does not tell you: how big the first steps are, and how the step size should end.

# Why Warmup

- Adam's step is the vector $\alpha\, \hat{m}/(\sqrt{\hat{v}}+\varepsilon)$, one coordinate at a time.
- At the first step $\hat{v}$ is a single gradient, so the update is close to $\alpha \cdot \text{sign}(g)$: every coordinate moves by roughly $\alpha$, before anything about the curvature is known.
- Warmup keeps $\alpha$ small until $\hat{v}$ and the local curvature are estimated, then raises it.
- This is the same statement as Lec 4's step size rule: stay below $2/L$ while $L$ is still changing.
- Warmup is a safety device, not a trick.

# Learning-Rate Schedules

- Constant α: the simplest choice, and the noise floor is proportional to α — the run stops improving once the gradient noise is of that size.
- Cosine decay: α falls to about 0 over the run. The noise floor vanishes, at the price of a small step size at the end.
- Warmup + cosine is the default recipe for pre-training a Transformer.
- Warmup–stable–decay holds α constant and decays only at the end, which makes a run easier to extend.
- GPT-3: warmup over the first 375M tokens, then cosine decay to 10% of the peak.

# Part 8 Summary

What to remember, the references, and where to find the code.

# Summary (1/2): Methods, Convergence, Curvature

- 1 AdaGrad, RMSProp and Adam are Gauss-Newton approximations; SGD and momentum are where the ladder starts.
- 2 Adam converges without modifying its update rule; a small $\beta 2$ can diverge.
- 3 Transformers need Adam because their Hessian is block-heterogeneous.

# Summary (2/2): Adam-mini, Muon, and What Is Next

- 4 The same theory gives Adam-mini: partition along Hessian blocks, about half the optimizer memory, one line of code.
- 5 Preconditioning rises from vectors to matrices (Shampoo → Moonlight → Muon); Muon needs the right learning rate and weight decay to work at scale.
- 6 Next lecture: optimization meets representation — Muon acts on the gradients, preconditioning acts on the weights.

# Mainly based on:

- DDA6202 by Ruoyu Sun, Lecture notes, CUHK-SZ
- Zhang, Chen, Shi, Sun*, & Luo; Adam Can Converge Without Any Modification on Update Rules, NeurIPS 22 (Spotlight)
- Zhang, Chen, Ding, Li, **Sun***, & Luo; Why Transformers Need Adam: A Hessian Perspective, NeurIPS 24
- Zhang*, Chen*, Li, Ding, Chen, Kingma, Ye, Luo & **Sun***; Adam-mini: Use Fewer Learning Rate To Gain More, ICLR 25

Yushun Zhang (currently @deepseek)



Congliang Chen



D.P. Kingma (Anthropic)



Yinyu Ye



Zhi-Quan Luo



Ruoyu Sun



# References on Neural Network Optimization

**For a big picture on neural network optimization, see:**

- **Book in preparation:**
  **R. Sun**, "Lectures on deep learning optimization", in preparation. To be published by **Springer Nature**.

**Surveys:**

- [S1] **R. Sun**, D. Li, S. Liang, T. Ding, R. Srikant, "The Global landscape of neural networks: An overview." **IEEE Signal Processing Magazine**, 2020
- **[S2] R. Sun**, "Optimization for deep learning: an overview." (survey paper). **Journal of Operations Research Society of China,** Special Issue on "Continuous Optimization: Past, Present and Future" (invited). 2020.

# Thank You for Listening!

Email 邮箱: sunruoyu@cuhk.edu.cn

Website 个人网站: ruoyus.github.io

**招聘 (at CUHK-SZ, SLAI, SRIBD): >50 个职位**

Faculty, research scientists, postdocs, visiting students, engineers…

方向: 1）大模型算法设计和分析;
2）大模型 Infra 和算子生成;
3）大模型垂域应用（通信网络，医疗，法治登）
4）强化学习和智能体

# Backup · not presented

The pages that follow are the parts of the workshop lecture this cut does not present: extra evidence, variants, and theory detail.

They are kept here so that nothing is ever lost — drag any of them forward if a question asks for it.

# Supervised Learning Loss

We have seen linear regression: $\min_w \sum_i (a_i^T w - b_i)^2$.

In machine learning, a common problem is

$$\min_w \sum_i \ell(y_i, f_w(x_i)),$$

where $l$ is a loss function, and $f_w(x)$ is a neural network.

Can set $F_i(w) = \ell(y_i, f_w(x_i))$, the loss for one sample.

More commonly, set $F_i(w)$ to be the loss for a mini-batch of samples.

# Exact Version of Adam: With Bias-correction term

[Kingma, Ba'15] added bias-correction term:

$m_0 \leftarrow 0$ (Initialize 1st moment vector)
$v_0 \leftarrow 0$ (Initialize 2nd moment vector)
$t \leftarrow 0$ (Initialize timestep)
**while** $\theta_t$ not converged **do**
  $t \leftarrow t + 1$
  $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep $t$)
  $m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)
  $v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)
  $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)
  $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)
  $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ (Update parameters)
**end while**
**return** $\theta_t$ (Resulting parameters)

Useful for the first few epochs; in theory, not affecting later convergence.

- For simplicity, we use (8), i.e., without bias-correction.

# Algorithms in PyTorch

Algorithms in torch.optim can be divided into three classes.

First, **SGD** (including momentum).

- SGD
- ASGD

Second, **LBFGS.** (famous nonlinear optimization method).

Third, **adaptive gradient methods**.

- Adagrad, RMSProp, Adam (including AMSGrad), Adadelta
- AdamW, Adamax, SparseAdam,

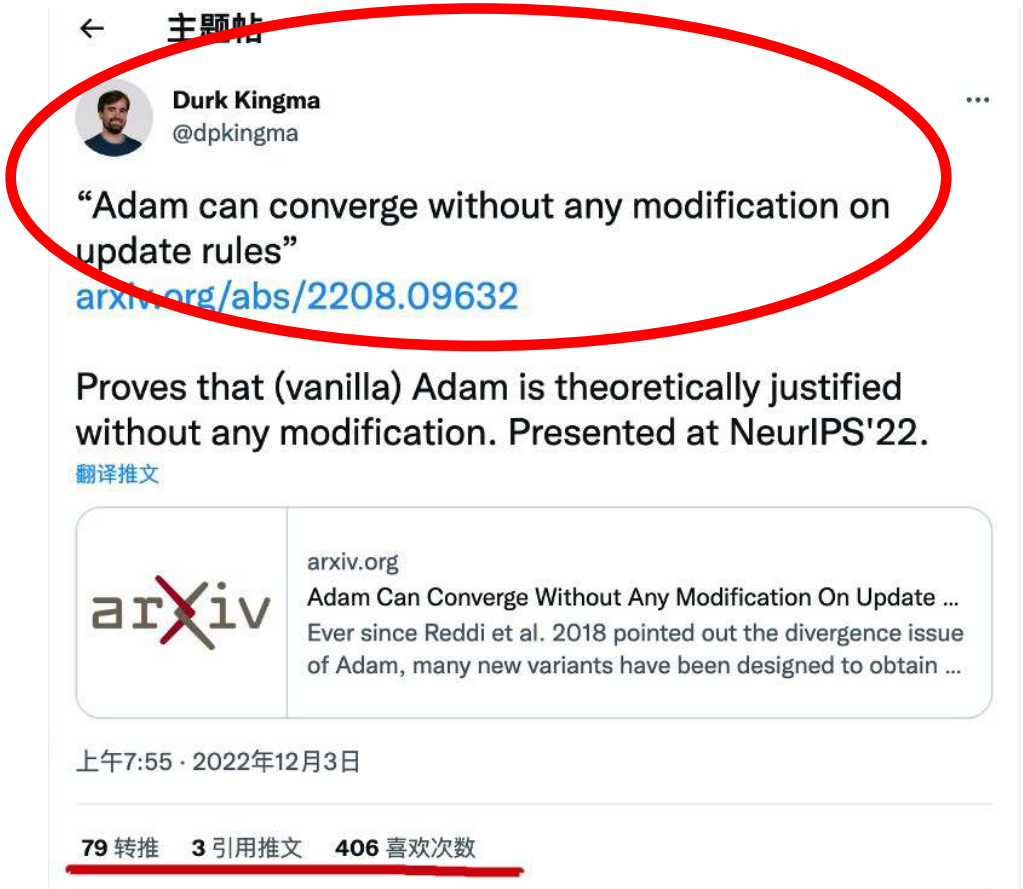
# Adam Variants

Brief intro of these adaptive gradient methods:

- Adadelta: related to RMSProp, but more complicated numerator
- AdamW: "correct" way to implement weight decay and Adam together.
- Adamax: a variant of Adam based on infinity norm
- SparseAdam: Implements lazy version of Adam algorithm suitable for sparse tensors.
- Adam-mini: use block-wise mean to replace coordinate-wise lr, Zhang et al., 2024.

# Our work is tweeted by **Dr. Kingma** (1st author of Adam)

Dr. Durk Kingma: inventor of Adam and VAE; co-founder of OpenAI; now a leader of Google Brain







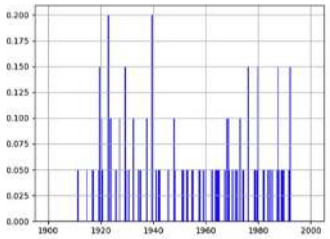


# Hetereogeneity makes SGD worse: Quadratic Prob

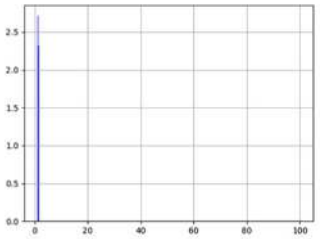
- Case study on quadratic models

$\min_w \frac{1}{2} w^T H w$, $H = diag(H1, H2, H3, H4)$, $H$ is PD

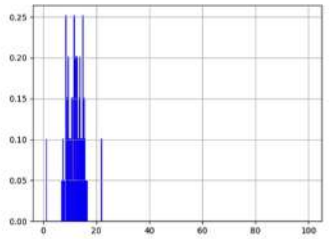
Case 1: $H$ with Transformer-type spectra: sampled from GPT2



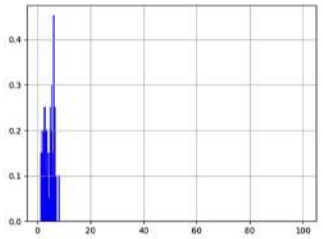
(a) Spectrum of $H_1$



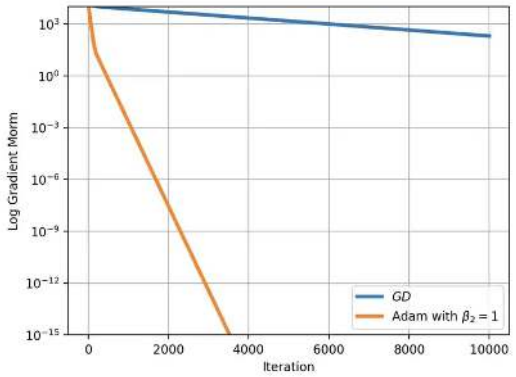
(b) Spectrum of $H_2$



(c) Spectrum of $H_3$



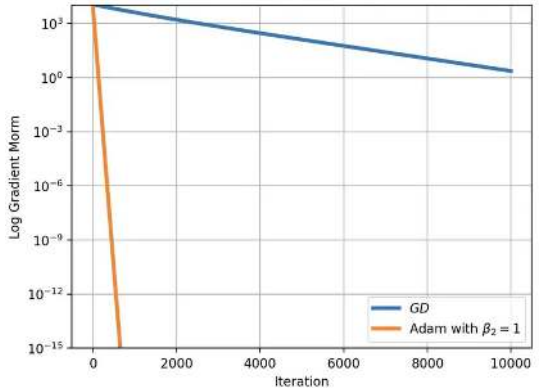
(d) Spectrum of $H_4$




GD is slower than Adam

Case 2: $H$ with simplified heterogeneous spectra

Eigenvalues of $H_I$ : { 1, 2, 3 } , { 99, 100, 101 } , { 1998,




GD is slower than Adam

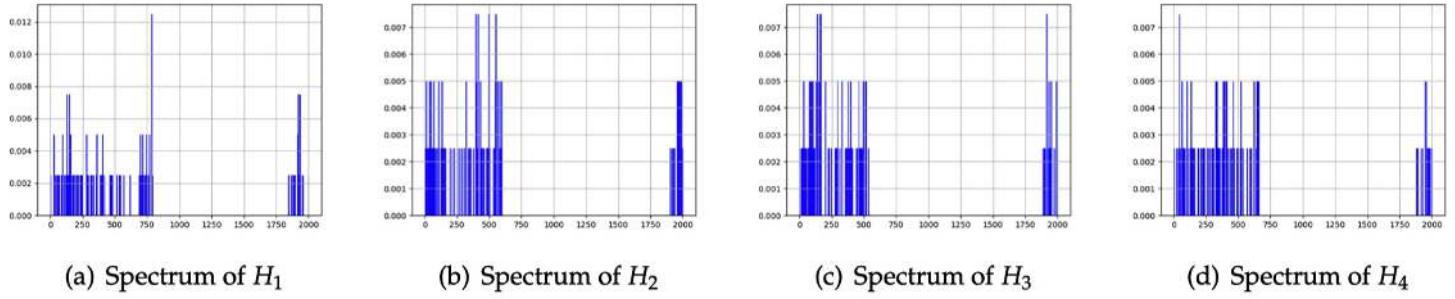
Remark:
Same condition number for case 1 & 2
but t e performance is different, due to homo & heterogeneity

# Homogeneity can make SGD work: (quadratic prob)

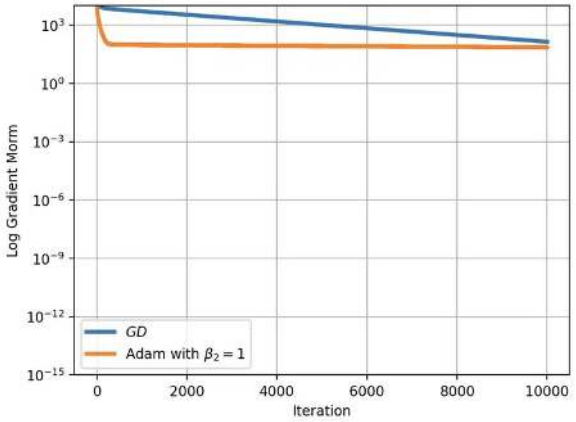
- Case study on quadratic models

$$\min_w \frac{1}{2} w^T H w,\ H = diag(H1, H2, H3),\ H \text{ is PD},\qquad H_l \in R^{3\times 3},\ l = 1,2$$

Case 3: $H$ with CNN-type spectra: sampled from ResNet18



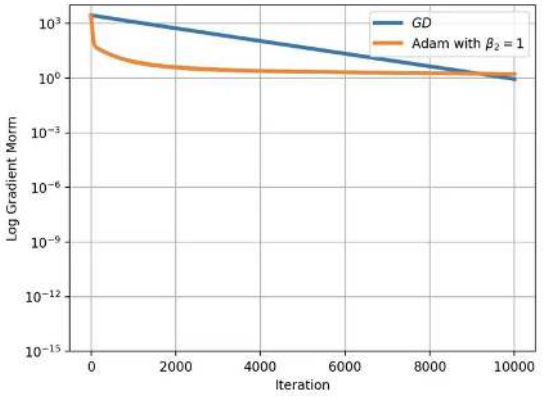
(a) Spectrum of $H_1$ (b) Spectrum of $H_2$ (c) Spectrum of $H_3$ (d) Spectrum of $H_4$




GD is similar as Adam

Case 4: $H$ with simplified homogeneous spectra

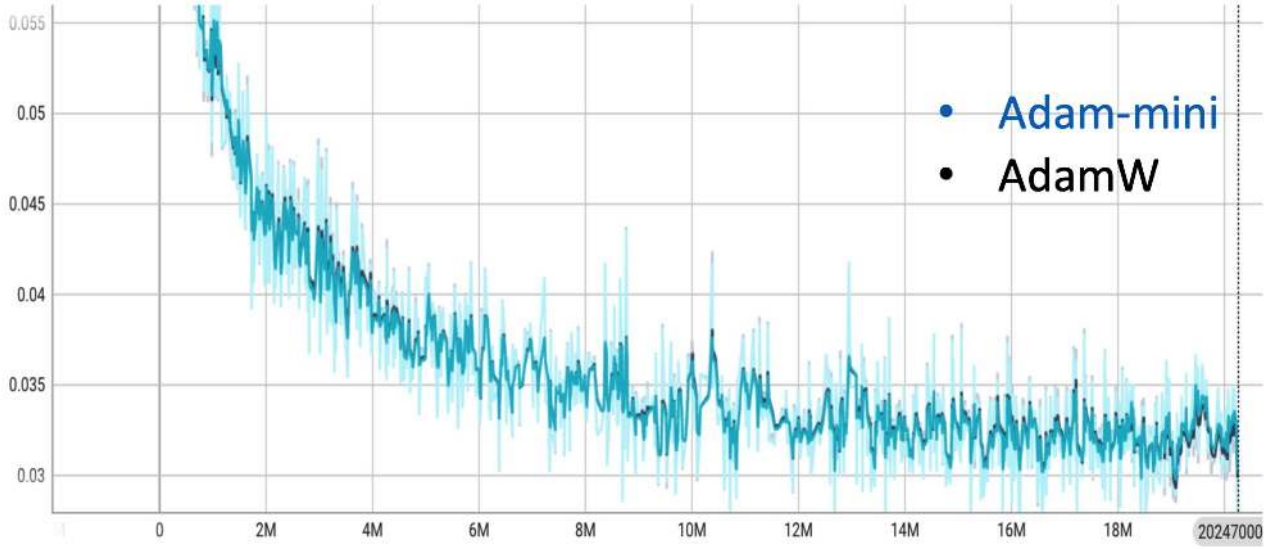
Eigenvalues of $H_l$ : { 1, 99, 1998 } , { 2, 100, 1999 } , { 3, 101, 2000 }




GD is similar as Adam

Remark:
Same condition number for case 1 & 3
All eigenvalues are t e same for case 2 & 4
but t e performance is different, due to homo & heterogeneity

# Diffusion models




**Diffusion model training:**
Loss vs. iteration




Independent verifier on twitter:
SDXL: Stable Diffusion XL (sized 2.6B)
One of t e SOTA Diffusion model

**Adam-mini** performs slightly better than **AdamW**, with **50%** less memory

# Summary of This Session

- We provide a derivation from Gauss-Newton method to Adam

- We explain that vanilla Adam converges when $\beta_2$ is large enough

- We provide an explanation why Transformer needs Adam, not SGD
  Hetereogeneity of block-Hessian-spectra causes SGD to underperform Adam

- We introduce a 50%-memory-saving variant of Adam: Adam-mini
  **Two features**: Principled block partition;  block-mean of v

# Moonlight paper of kimi

## MUON IS SCALABLE FOR LLM TRAINING

TECHNICAL REPORT

**Jingyuan Liu**[1] **Jianlin Su**[1] **Xingcheng Yao**[2] **Zhejun Jiang**[1] **Guokun Lai**[1] **Yulun Du**[1]
**Yidao Qin**[1] **Weixin Xu**[1] **Enzhe Lu**[1] **Junjie Yan**[1] **Yanru Chen**[1] **Huabin Zheng**[1]
**Yibo Liu**[1] **Shaowei Liu**[1] **Bohong Yin**[1] **Weiran He**[1] **Han Zhu**[1] **Yuzhi Wang**[1]
**Jianzhou Wang**[1] **Mengnan Dong**[1] **Zheng Zhang**[1] **Yongsheng Kang**[1] **Hao Zhang**[1]
**Xinran Xu**[1] **Yutao Zhang**[1] **Yuxin Wu**[1] **Xinyu Zhou**[1] * **Zhilin Yang**[1]

[1] Moonshot AI [2] UCLA

# 如何计算 $UV^T$? Appendix A of [B&N]

如何根据 W = $U\Sigma V^T$ 来计算$UV^T$?
[Bernstein & Newhouse, Appendix A] 顺便提了几个方法:

1) Perform SVD, discard Σ, keep U, V.

2) Sketching. 随机低维投影法来算 SVD (Martinsson and Tropp, 2020)

3) Newton Iteration. $(GG^T)^{-1/2}$ 可用Newton iteration
(Lakić, 1998), Higham (2008)'s book, Anil et al. 2020.

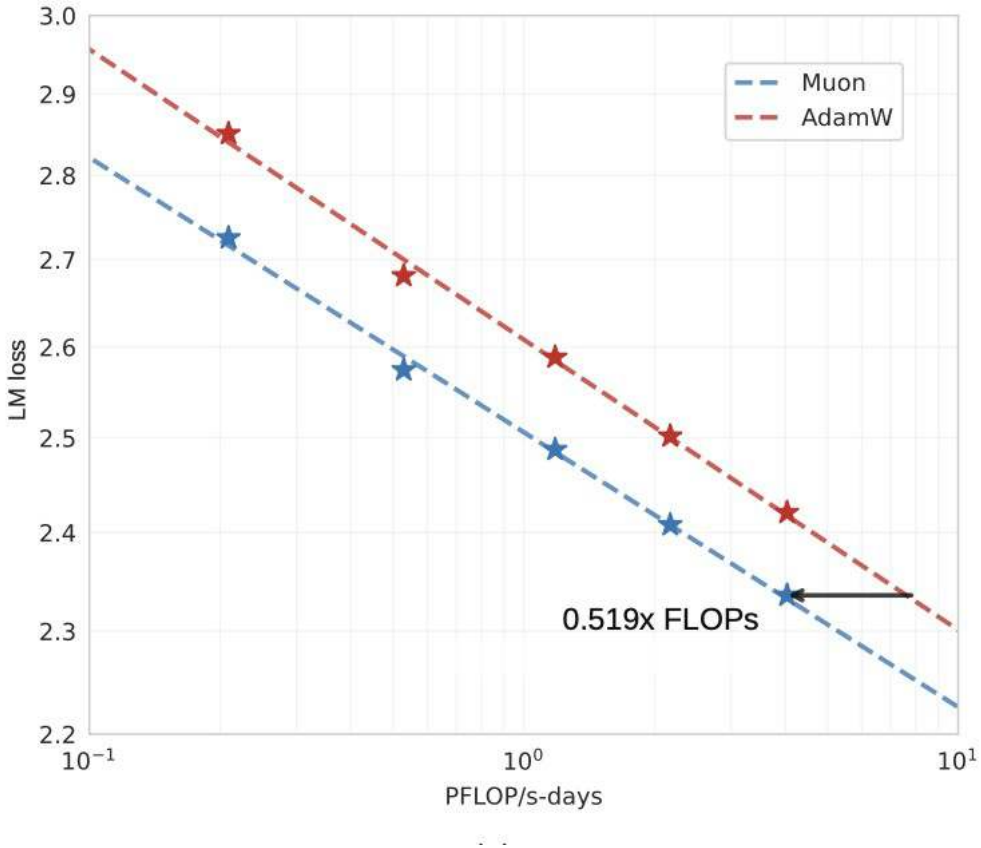
**4) Newton-Schulz iteration (见下页)**

# Kimi 版本的 Muon: 声称 2 倍速度

16B 的 MoE 上训练

同样的 loss,
52% 的 FLOPS

加速 2 倍



# Part III　其他进展

# Muon As Preconditioned Gradient Method

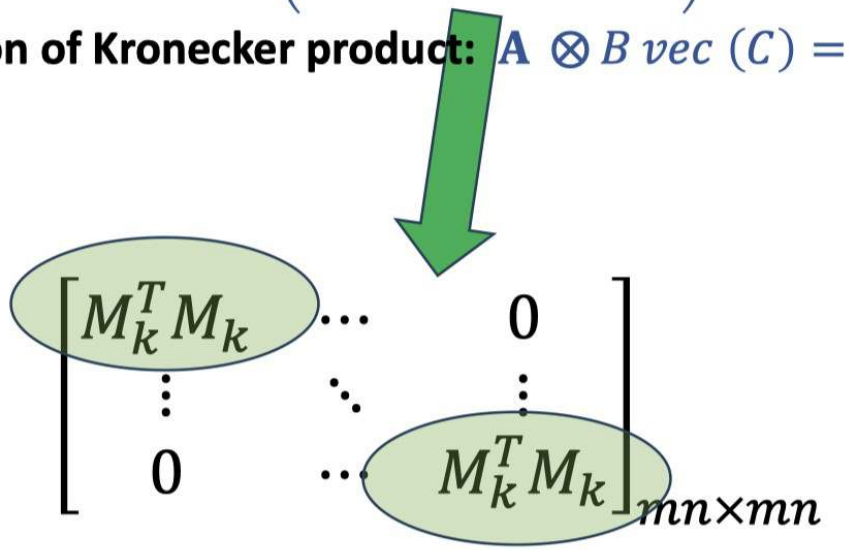
- **Matrix form of Muon:** $W \in R^{m \times n}$

$$W_{k+1} = W_k - \eta_k \left(M_k M_k^T\right)^{-\frac{1}{2}} M_k = W_k - \eta_k \, I_m \, M_k \left(M_k^T M_k\right)^{-\frac{1}{2}}$$

- **Flattened vector form of Muon:**

$$\mathrm{vec}(W_{k+1}) = vec(W_k) - \eta_k \left( I_m \bigotimes \left(M_k^T M_k\right) \right)^{-\frac{1}{2}} vec(M_k)$$

**We used the proposition of Kronecker product:** $\mathbf{A} \otimes B \; vec\,(C) = ACB$



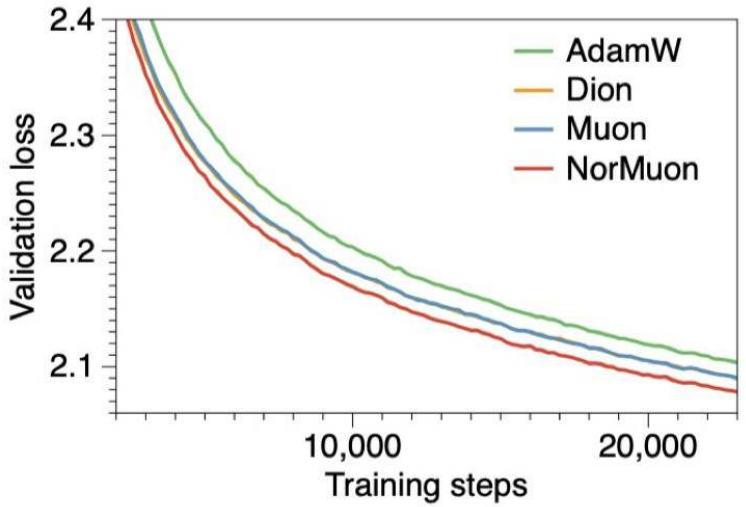

**Key observation:**
Muon use the **same** preconditioner across neurons in the same layer

**But neurons in Transformers are heterogeneous… [1, 2]**
**How to improve Muon?**


[1] **Why Transformers Need Adam: A Hessian Perspective**, NeurIPS 2024
[2] Technical report of Aurora optimizer, May, 2026

# NorMuon: Adam-mini + Muon

- **NorMuon:** add neuronwise lr to Muon
- by Microsoft and GaTech, including an author of LoRA




(a) Pretraining results of 1.1B model.

**Algorithm 1** NorMuon

**Input:** Initial weights $\mathbf{W}_0 \in \mathbb{R}^{m\times n}$, loss $L$, learning rate $\eta$, momentum parameters $(\beta_1, \beta_2)$, perturbation parameter $\varepsilon$, weight decay $\lambda$.
Initialize $\mathbf{M}_0 \in \mathbb{R}^{m\times n} \leftarrow \mathbf{0}$, $\mathbf{v}_0 \in \mathbb{R}^m \leftarrow \mathbf{0}$
**for** $t = 1, 2, \ldots$ **do**
  $\mathbf{G}_t \leftarrow \nabla_{\mathbf{W}} L(\mathbf{W}_t)$
  $\mathbf{M}_t \leftarrow \beta_1 \mathbf{M}_{t-1} + (1-\beta_1)\mathbf{G}_t$
  $\mathbf{O}_t \leftarrow \mathrm{NS5}(\mathbf{M}_t)$
  $\mathbf{v}_t \leftarrow \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathrm{mean}_{\mathrm{cols}}(\mathbf{O}_t \odot \mathbf{O}_t)$
  $\mathbf{V}_t \leftarrow \mathrm{ExpandRows}(\mathbf{v}_t)$
  $\widehat{\mathbf{O}}_t \leftarrow \mathbf{O}_t \oslash \left(\sqrt{\mathbf{V}_t} + \varepsilon\right)$
  $\hat{\eta} = 0.2\eta\sqrt{mn}/\|\widehat{\mathbf{O}}_t\|_F$
  $\mathbf{W}_{t+1} \leftarrow \mathbf{W}_t - \eta\lambda\mathbf{W}_t - \hat{\eta}\widehat{\mathbf{O}}_t$
**end for**

## NorMuon: Making Muon more efficient and scalable

Zichong Li [*1], Liming Liu [*1 †], Chen Liang [2], Weizhu Chen [2], Tuo Zhao [1]
[1]Georgia Tech [2]Microsoft

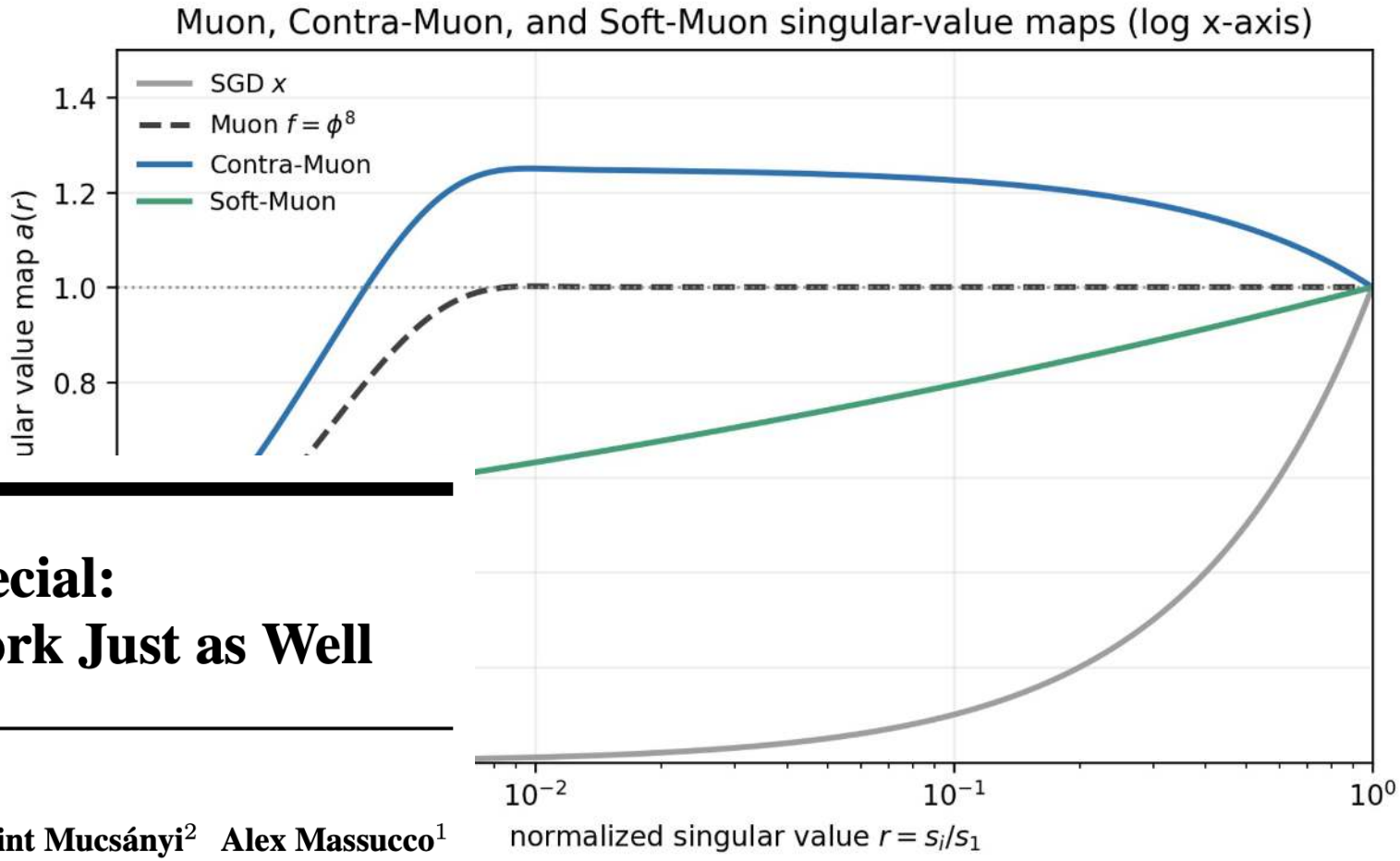
This observation motivates our key insight: while Muon's orthogonalization effectively reduces the condition number of updates, the remaining high variance in neuron norms still creates an imbalanced learning dynamic, potentially leading to inefficient parameter usage. Drawing inspiration from Adam-mini's success Zhang et al. (2025) with per-neuron adaptive learning rates, we propose to incorporate second-order momentum to normalize these disparate scales and ensure more balanced parameter updates. Our method, **NorMuon**, augments Muon's orthogonalization with neuron-wise adaptive learning rates computed from accumulated second-order statistics.

# Contra-Muon and Soft-Muon

Contra-Muon
抬升中间奇异值

Soft-Muon
Linear combination of NS iterations
（图中只是个特例）

$$g(x) = \alpha_0 x + \alpha_1 \varphi(x) + \ldots \alpha_T \varphi(x)$$




**Muon is Not That Special:**
**Random or Inverted Spectra Work Just as Well**


**Zakhar Shumaylov**[1] **Nathaël Da Costa**[2] **Peter Zaika**[1] **Bálint Mucsányi**[2] **Alex Massucco**[1]
**Yoav Gelberg**[3] **Carola-Bibiane Schönlieb**[1] **Yarin Gal**[3] **Philipp Hennig**[2]

[1]University of Cambridge [2]University of Tübingen [3]University of Oxford


ttps://nilin.git ub.io/contra-muon-and-soft-muon/

# Aurora: A Leverage-Aware Optimizer for Rectangular Matri

5.05.202

Alec Dewulf*, Dhruv Pai*, Li Yang*,
* Core Contributor; Correspondence to alec@tilde

Prevent neuron death for tall matrices
(e.g. up and gate proj in MLP)

Related to
block Hessian
Heterogeneity?

1. Low gradient norm | 2. Low update norm | 3. Persistence over training

Muon optimizer

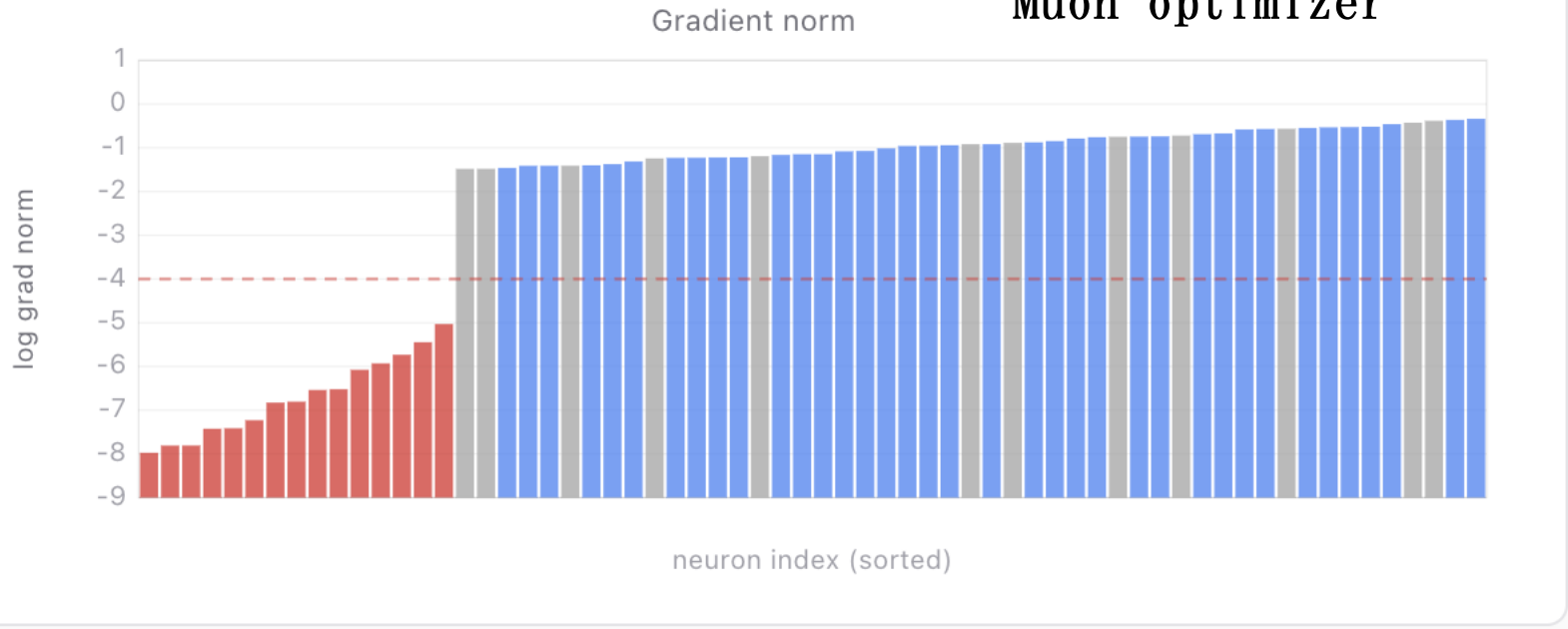
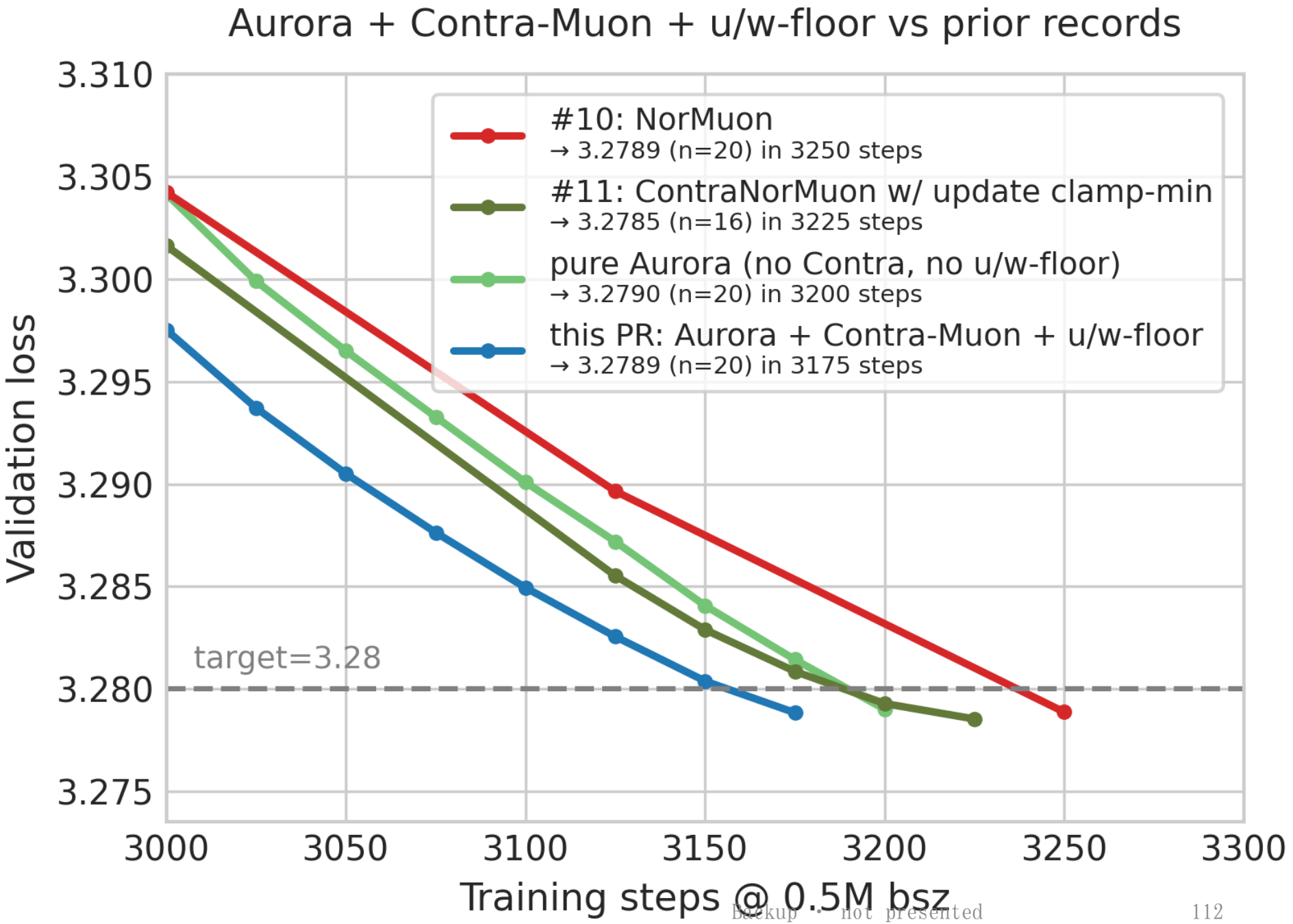



Figure 5. Visualizing each criterion for identifying dead neurons: low effective gradient norm, low effective update norm, and persistence of both over training. We identify a neuron as dead if it satisfies all three criteria.




May 8

modded-nanogpt benchmark

# Row-normalization

- **Insights related to Adam-mini**

- SCION
- SCALE (Hong Mingyi)
- NorMuon
- REG
- RMNP
- Mano
- Nora
- Aurora
- Muown （出现在 modded-nanogpt 榜单前列）
- …

1. **NorMuon**：最直接；Aurora 的问题设定基本从它来。
2. **MuonEq**：也是 Muon-family，而且明确 row/column normalization，只是放在 orthogonalization 前。
3. **Gradient Multi-Normalization / SinkGD**：最像“交替 row/column normalization”的算法思想。
4. **Width Scaling / MOGA**：如果你关心理论动机，尤其是 width scaling / operator norm。
5. **SRON / RMNP / SCALE / SWAN**：分别覆盖 row-only、row replacing Muon、column-only、row+whitening 这些变体。

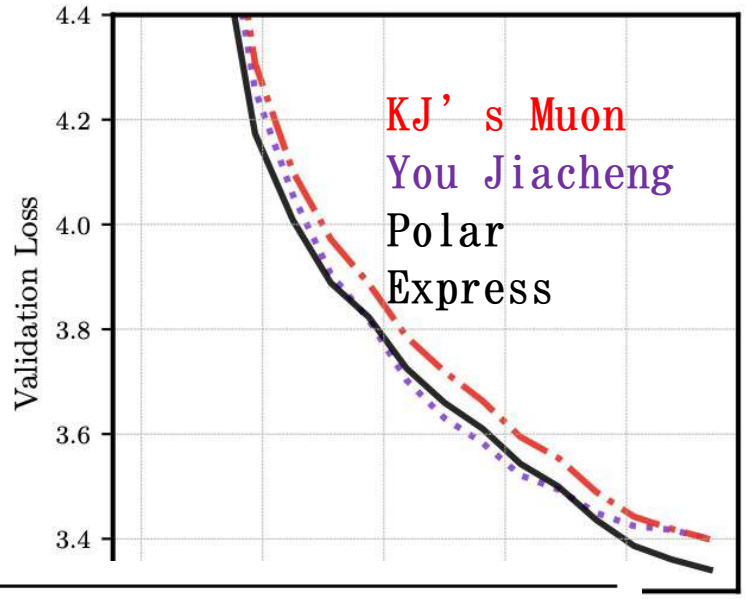
# THE POLAR EXPRESS: OPTIMAL MATRIX SIGN METHODS AND THEIR APPLICATION TO THE MUON ALGORITHM

**Noah Amsel**
New York University
noah.amsel@nyu.edu

**David Persson**
New York University
Flatiron Institute
dup210@nyu.edu

**Christopher Musco**
New York University
cmusco@nyu.edu

**Robert M. Gower**
Flatiron Institute
rgower@flatironinstitute.org




**Accelerating Newton-Schulz Iteration for Orthogonalization via Chebyshev-type Polynomials**

**Ekaterina Grishina** [*1] **Matvey Smirnov** [*1] **Maxim Rakhuba** [1]

CANS

# Squeezing 1-2% Efficiency Gains Out of Muon by Optimizing the Newton-Schulz Coefficients

February 21, 2025 · 8 min · Franz Louis Cesista · Crossposted from X (formerly Twitter)

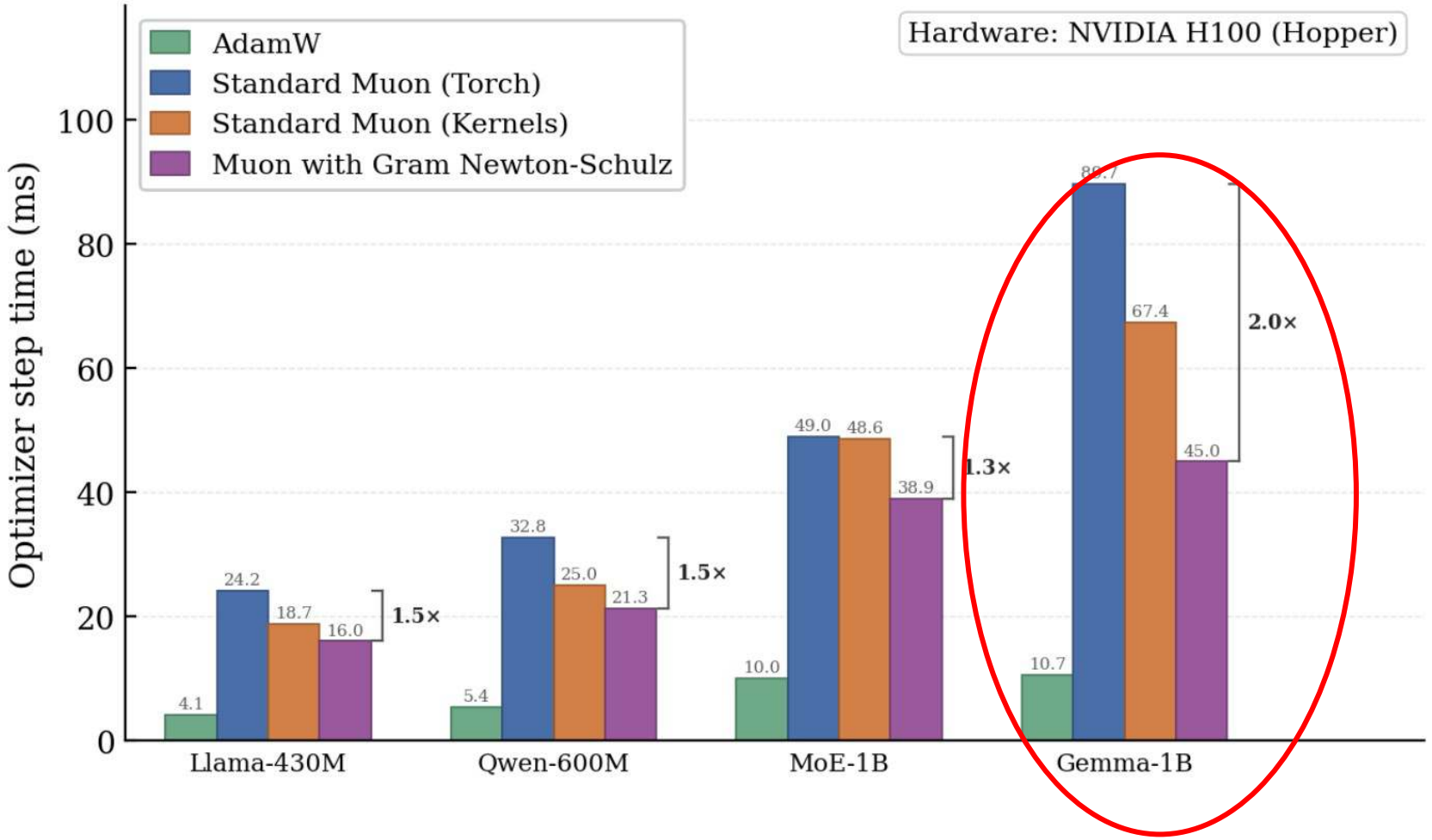
Quinti

https://leloykun.github.io/ponder/muon-opt-coeffs/

Ji-Ha Kim's blog: https://jiha-kim.github.io/posts/rational-polar-decomposition/

Rational function approximation

# Speedup of Optimizer Step




2x speedup of each optimizer step

*Figure 21: Hopper architecture end-to-end optimizer step during training, including matrix splitting and recombination for QKV and MLP, LR scaling, master weight updates, and the scalar optimizer (AdamW) step for non-2D weights.*